

Transformers

cnrs

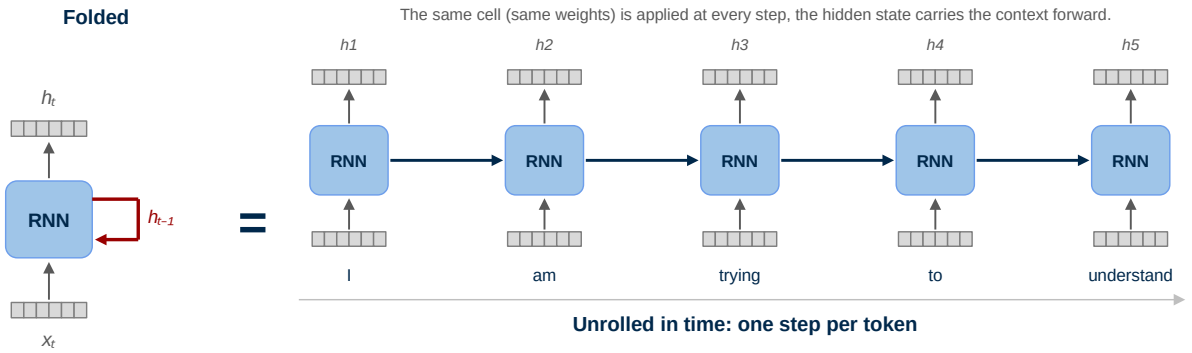
CC BY NC SA



From RNN to LLM

A long time ago: recurrent neural networks

Recurrent neural network (RNN) : reads a sequence one token at a time and updates a hidden state that summarizes everything read



- **Sequential:** step t must wait for step t − 1, so training cannot be parallelized over the sequence.
- **Short memory:** information from distant tokens fades (vanishing gradients).

What the slide shows

A recurrent neural network (RNN) drawn in two equivalent ways.

- On the left, the "folded" form: a single RNN cell receives the current word x■ and produces the hidden state h■. The red loop sends this state back into the cell, where it becomes the input of the next step (h■■■). This is the classic way of drawing an RNN (for example in C. Olah, "Understanding LSTM Networks", 2015).
- On the right, after the "=" sign, the same cell "unrolled" in time over the sentence "I am trying to understand": one copy per word, with the same weights at every step. At the bottom, each word becomes a vector; at the top, h1 to h5 are the successive hidden states. The horizontal arrows are the loop of the folded form, unfolded step by step.

The principle

At step t, the cell combines the current word x■ and the previous state h■■■ to produce h■ = f(h■■■, x■). The hidden state is a fixed-size vector that acts as memory: it must summarize everything read so far. The last state, h5, is therefore supposed to represent the whole sentence.

Why two drawings

The folded form shows that there is only one set of weights, reused for every word. The unrolled form shows the actual computation, step by step: it is the one that reveals the two limitations.

The two limitations (bottom of the slide)

- Sequential: h3 can only be computed after h2. The computation cannot be parallelized along the sentence, which makes training slow on GPUs, which are designed to run many computations at the same time.
- Short memory: the information from a distant word has to go through every intermediate step. During training, the gradient is multiplied at each step and vanishes (vanishing gradient, Bengio et al., 1994). The network learns long-range dependencies poorly.

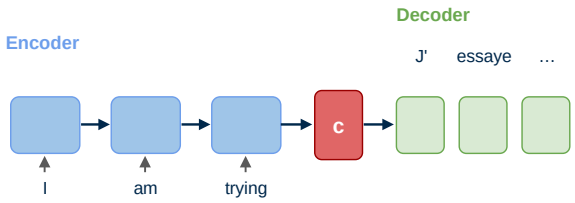
Context

Simple RNNs date back to the 1980s-1990s (Elman, 1990). LSTMs (Hochreiter & Schmidhuber, 1997) and GRUs (Cho et al., 2014) add "gates" that control what the state keeps or forgets. They reduce the memory problem without removing it, and remain sequential.

Seq2seq and the first attention (2014)

Attention : at each output step, the decoder takes a weighted average of all encoder states instead of relying on one fixed vector.

1 · The bottleneck: one fixed vector



Encoder–decoder RNN (Sutskever et al., 2014). The encoder squeezes the English sentence into one vector **c**.

The decoder must write the whole French sentence from **c** alone: quality drops on long sentences.

2 · Attention: the decoder looks back at every word

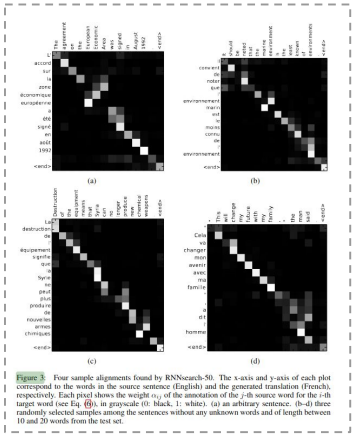


Figure 3 Four sample alignments found by RNNsearch-50. The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j-th source word for the i-th target word (see Eq. (6)), in grayscale (0: black, 1: white). (a) an arbitrary sentence. (b–d) three randomly selected samples among the sentences without any unknown words and of length between 10 and 20 words from the test set.

What the paper found. Each square is the weight between an English word (column) and the French word being written (row): white = 1, black = 0.

Mostly diagonal: words are translated in order.

Off the diagonal: in (a), “European Economic Area” becomes “zone économique européenne”: the model looks at the words in reverse order.

No alignment labels: the weights are learned from translation alone.

Bahdanau et al. (2014), “Neural Machine Translation by Jointly Learning to Align and Translate”, Figure 3
https://arxiv.org/abs/1409.0473

What the slide shows

The slide is split in two by a dashed line: the problem on the left, the solution on the right.

1 · The bottleneck (left)

An RNN-based encoder-decoder architecture (seq2seq, Sutskever et al. and Cho et al., 2014), used for translation. The encoder (blue boxes) reads the source sentence "I am trying". Everything it has read is summarized in a single vector c (red box). The decoder (green boxes) generates the translation word by word ("J'", "essaye", …) from this single vector. Whatever the length of the sentence, it must fit into a fixed-size vector: on long sentences, information is lost and translation quality drops sharply.

2 · Attention (right, Bahdanau, Cho and Bengio, 2014)

Instead of a single vector c, the decoder keeps access to all the encoder states, one per source word. For each word it generates, it computes a relevance score for each source word, turns these scores into weights (softmax), then takes the weighted average of the encoder states. It thus "looks at" the useful source words at the moment it needs them.

What the paper found (figure 3)

- Each matrix crosses the English words (columns) with the generated French words (rows). Each cell is the weight given to an English word when writing a French word: white = 1, black = 0. We can therefore read directly which source words the model considers important for each word it produces.
- The diagonal dominates: most of the time, words are translated in the same order.
- The deviations from the diagonal are the real finding. In matrix (a), "European Economic Area" becomes "zone économique européenne": when writing "zone", the model looks at "Area", then goes back to "Economic" and "European". It found the word-order inversion between English and French on its own.
- A French word can also rely on several English words at once, for example to choose the article or the agreement.
- No alignment was provided: these weights are learned from translated sentence pairs alone.

Why it matters

This is the idea that will be generalized in 2017: attention becomes the central mechanism, and RNNs disappear.

2017: Attention Is All You Need

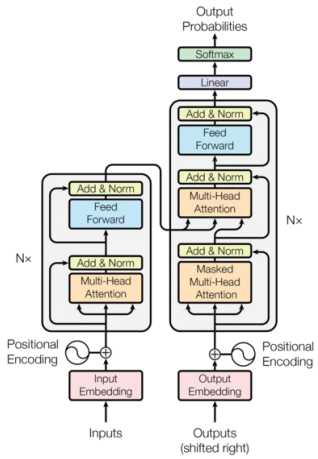


Figure 1 of Vaswani et al., NeurIPS 2017 (arXiv:1706.03762)

- **No recurrence:** all tokens are processed in parallel.
- Any token reaches any other in a single step.
- **Base model :** encoder–decoder, 6 + 6 layers, 65 M parameters (base model).

What the slide shows

Figure 1 of the paper "Attention Is All You Need" (Vaswani et al., Google, NeurIPS 2017): the full architecture of the original Transformer. On the left, the encoder; on the right, the decoder. Each one is a stack of N = 6 identical blocks ("Nx"). The encoder block contains a multi-head attention followed by a feed-forward network, each followed by "Add & Norm" (residual connection and normalization). The decoder block additionally contains a masked attention (it cannot see future words) and an attention over the encoder output (cross-attention). At the bottom, the token embeddings, to which a positional encoding is added (the sinusoidal symbol). At the top right, a linear layer and a softmax give the probability of the next word. All these building blocks are detailed later in the course.

The three points on the right

- No more recurrence: all the tokens of a sentence are processed at the same time. Training can be parallelized on GPUs, which made it possible to train much larger models on much more data.
- Any token reaches any other token in a single attention step, whereas in an RNN the information has to go through every intermediate position. Long-range dependencies become easy to learn.
- The model was designed for translation: encoder-decoder, 6 + 6 layers, vectors of size 512, 8 attention heads, 65 million parameters for the "base" model (213 M for "big").

Results at the time

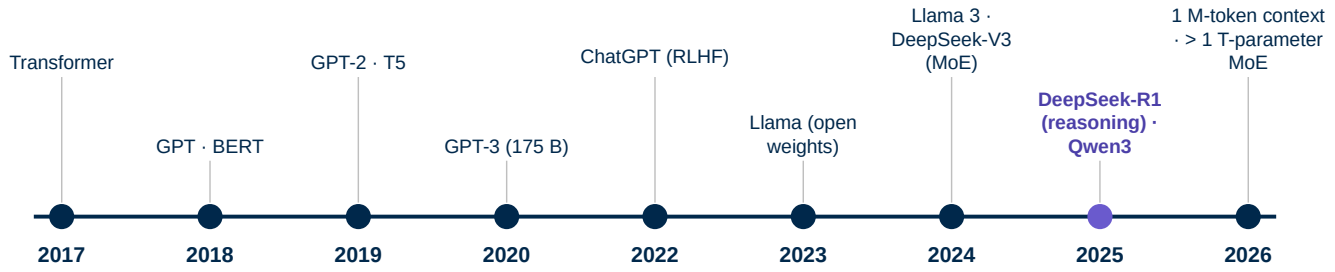
Best scores on English-German and English-French translation (WMT 2014), for a much lower training cost: 12 hours on 8 GPUs for the base model.

The trade-off

Attention compares each token with all the others: its cost grows with the square of the sequence length.

From the Transformer to today's LLMs

Large language model (LLM) a Transformer trained on trillions of tokens to predict the next token, then adapted to follow instructions.



What the slide shows

A timeline of the steps leading from the 2017 Transformer to today's LLMs. The purple dot in 2025 marks Qwen3, our running example.

Reading the timeline

- 2017: the Transformer, for translation.
- 2018: the two families split. GPT (OpenAI) keeps the decoder and learns to predict the next word. BERT (Google) keeps the encoder and learns to recover masked words.
- 2019: GPT-2 (1.5 billion parameters) shows that a language model can generate coherent text. T5 (Google) frames every task as "text in, text out".
- 2020: GPT-3 (175 billion parameters) can perform a task from a few examples given in the prompt, without retraining.
- 2022: InstructGPT, then ChatGPT (November). The model is tuned to follow instructions, using reinforcement learning from human feedback (RLHF).
- 2023: Llama (Meta) popularizes open-weight models, which can be downloaded and modified.
- 2024: Llama 3 is trained on 15 trillion tokens. DeepSeek-V3 brings Mixture of Experts into the mainstream: 671 billion parameters, of which 37 billion are used per token.
- 2025: DeepSeek-R1 is an open-weight reasoning model trained with reinforcement learning. Qwen3 (Alibaba) offers a family of open models with "thinking" and "direct answer" modes.
- 2026: context windows of one million tokens and MoEs with more than 1,000 billion parameters (DeepSeek V4, Kimi K3).

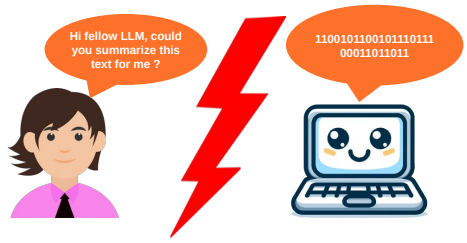
Definition

An LLM is a Transformer, usually a decoder, pre-trained on trillions of tokens to predict the next token, then adapted to follow instructions. The core architecture has changed little since 2017: it is mostly scale (data, parameters, compute) and post-training that have transformed its capabilities.



Introduction

The problem of NLP: Sparse data



Traditional NLP (until the 2010s)

1 · Words split with hand-written rules → **2 · Normalize** lowercase, stop words, stems → **3 · Count** bag of words, TF-IDF → **4 · Classify** logistic regression, SVM

Documents × vocabulary: a huge, empty matrix

	aardvark	…	cat	…	dog	…	king	…	queen	…	sleep	…	zoo
doc 1	0		1		0		0		0		1		0
doc 2	0		0		2		0		0		0		0
doc 3	0		0		0		1		1		0		0
doc 4	0		1		0		0		0		0		0
doc 5	0		0		0		0		0		3		0

One-hot words

king	0	0	1	0	0	0	…
queen	0	0	0	1	0	0	…
mercenary	0	0	0	0	0	1	…

The main problems

- Huge and almost empty
- No meaning: king is as far from queen as from mercenary
- Word order is lost
- Unknown words become <UNK>

What the slide shows

At the top left, the illustration of the problem: a person writes in natural language, the computer only "sees" 0s and 1s. A neural network only handles numbers, so text has to be turned into numbers. The rest of the slide shows how traditional NLP did this, and why it was a problem.

Traditional NLP (until the 2010s), in 4 steps

- 1 · Words: split the text into words with hand-written rules (spaces, punctuation, special cases).
- 2 · Normalize: lowercase, remove stop words ("the", "are"), reduce words to their stem (stemming: "sleeping" → "sleep").
- 3 · Count: represent a document by the number of occurrences of each vocabulary word (bag of words), often weighted by TF-IDF, which gives more weight to words that are rare and characteristic of a document.
- 4 · Classify: feed these vectors to a classical classifier (logistic regression, SVM, Naive Bayes).

The example "The cats are sleeping" → cat, sleep: only one count per vocabulary word remains; word order and grammar are gone.

The documents × vocabulary matrix (bottom left)

One row per document, one column per vocabulary word, and in each cell the number of occurrences of the word in the document. Each document only uses a handful of words: almost every cell is 0. With a real vocabulary of more than 50,000 words, more than 99 % of the cells are empty. This is a sparse matrix.

One-hot vectors (center)

To represent a single word, it is given a vector the size of the vocabulary, filled with zeros except for a 1 at its position. "king", "queen" and "mercenary" each have a 1 in a different place: their vectors never overlap, so all words are at the same distance from each other.

The main problems (right)

- Huge and almost empty: expensive in memory and compute.
- No meaning: "king" is as far from "queen" as from "mercenary". The model cannot generalize from one word to a similar word.
- Word order is lost: "the dog bites the man" and "the man bites the dog" have the same bag of words.
- A word missing from the vocabulary has no representation: it becomes an "unknown" symbol (<UNK>).

What comes next

Tokenization

Tokenizer : splits text into tokens and maps each one to an integer ID. It is built once, before training the model.

Anti *- chocs,* ***constitution***
et actu - ***elle*** *-* ***ment*** *sont appris.*

Anticonstitutionnellement *sera compris.*

What the slide shows

An example of splitting a text into tokens. Frequent words or frequent pieces ("Anti", "constitution", "elle", "ment") have been learned as units. The very long and rare word "Anticonstitutionnellement" does not need to be in the vocabulary: it can be rebuilt by assembling these pieces.

The definition

The tokenizer splits text into tokens, often pieces of words, and assigns each token an integer ID. It is built once, before the model is trained, from the frequencies observed in a large corpus. It is then frozen: it is not part of the learned parameters, and changing the tokenizer would require retraining the model.

Why subwords

- With whole words, the vocabulary would be gigantic and any unknown word would be lost.
- With characters, sequences would be very long and each unit would carry little meaning.
- Subwords are a compromise: a fixed-size vocabulary (from a few tens to a few hundreds of thousands of tokens) that can write any word.

A caveat about the example

The split shown follows grammar for teaching purposes. A real tokenizer follows corpus frequencies, not the rules of the language: for example it splits "anticonstitutionnellement" into "ant / icon / stitution / nellement".

Possible demo

The Tokenizer Playground (huggingface.co/spaces/Xenova/the-tokenizer-playground) lets you type a sentence and see the split and the IDs for several real tokenizers.

Tokenizers

Model	Year	Algorithm	Vocabulary
BERT	2018	WordPiece	30,522
GPT-2	2019	Byte-level BPE	50,257
T5	2019	Unigram (SentencePiece)	32,000
Llama 2	2023	BPE (SentencePiece)	32,000
Llama 3	2024	Byte-level BPE	128,256
Qwen3	2025	Byte-level BPE	**151,669**
Gemma 3	2025	SentencePiece	≈ 262k

What the slide shows

The tokenizers of a few well-known models, with their year, algorithm and vocabulary size. The row for Qwen3, our running example, is highlighted.

The algorithms in the table

- WordPiece (BERT): Google's variant. It merges the pairs of pieces that appear much more often together than separately. Pieces inside a word are marked "##".
- BPE (Byte-Pair Encoding): repeatedly merges the most frequent pair of symbols. Detailed on the next slide.
- Byte-level BPE: the same BPE, but starting from the 256 possible bytes instead of characters. Any text can then be encoded, in any script, with emojis or code: there is no unknown token anymore.
- Unigram: the opposite logic, starting from a very large vocabulary and removing the least useful tokens.
- SentencePiece is not an algorithm but a Google library that implements BPE and Unigram.

The trend

Byte-level BPE has become the standard (GPT-2, Llama 3, Qwen, DeepSeek), and vocabularies have grown a lot: from 30,000-50,000 tokens in 2018-2019 to 128,000-262,000 today.

The trade-off

A large vocabulary shortens sequences: fewer tokens are needed for the same text, hence less computation in attention. In return, the embedding matrix grows (vocabulary × d). For Qwen3-8B: 151,669 tokens, i.e. about 622 million embedding parameters.

Sources

The models' papers and tokenizer files; Qwen3 technical report (arXiv:2505.09388).

BPE (Byte-Pair Encoding)

BPE : start from characters and repeatedly merge the most frequent adjacent pair into a new token.

1. Training corpus

Word	Count	Start
low	5	l o w
lower	2	l o w e r
newest	6	n e w e s t
widest	3	w i d e s t

2. Merge the most frequent pair, repeat

Step	Merge	Count	New token
1	e + s	9	es
2	es + t	9	est
3	l + o	7	lo
4	lo + w	7	low
5	n + e	6	ne
6	ne + w	6	new

3. Encode new words

Apply the merges, in the same order:

`lowest` → **low** **est**

`newer` → **new** **e** **r**

`slow` → **s** **low**

Sennrich et al., ACL 2016

What the slide shows

The BPE algorithm run on the small toy corpus from the paper by Sennrich et al. (2016), which introduced BPE for text. The merges shown were computed by a minimal implementation of the algorithm.

1. The training corpus

Four words with their counts: low (5), lower (2), newest (6), widest (3). At the start, each word is split into characters.

2. The merges

We count all pairs of adjacent symbols, taking word frequencies into account. "e + s" appears 9 times (6 in newest, 3 in widest): it is the most frequent pair. It is merged into a new token "es". We count again, then repeat: "es + t" (9), "l + o" (7), "lo + w" (7), "n + e" (6), "ne + w" (6). Each merge adds one token to the vocabulary. We stop when the desired vocabulary size is reached. At step 1, "s + t" also has 9 occurrences: in case of a tie, the implementation takes the first pair encountered.

3. Encoding a new word

The learned merges are applied in the same order. "lowest" was never seen, but it becomes "low" + "est". "newer" becomes "new" + "e" + "r". "slow" becomes "s" + "low".

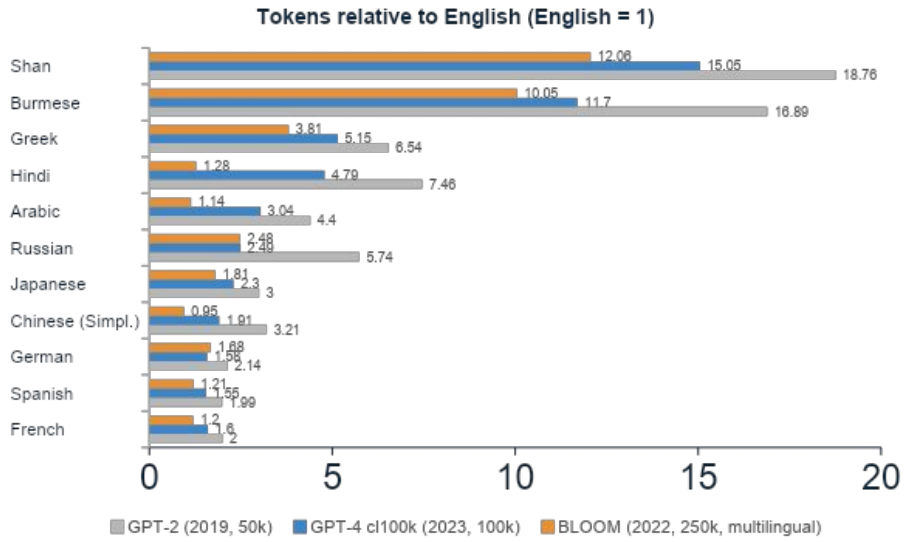
What training produces

A vocabulary (the initial symbols plus one token per merge) and the ordered list of merges. Encoding is deterministic: the same text always gives the same tokens. A real tokenizer learns 100,000 to 200,000 merges on terabytes of text, starting from the 256 bytes; the space is then attached to the following token ("■low").

Source

Sennrich, Haddow and Birch, "Neural Machine Translation of Rare Words with Subword Units", ACL 2016: aclanthology.org/P16-1162 (arXiv:1508.07909). BPE was originally a data compression algorithm (Gage, 1994).

Tokens are not equal across languages



More tokens means more compute, a higher cost per request and less text in the context window.

Data: Petrov et al., NeurIPS 2023, FLORES-200 (github.com/AleksandarPetrov/tokenization-fairness, MIT).

https://arxiv.org/pdf/2305.15425

What the slide shows

For the same sentence translated into different languages, the number of tokens needed relative to English (English = 1), with three tokenizers: GPT-2 (2019, 50,000 tokens, grey), GPT-4's "cl100k" (2023, 100,000 tokens, blue) and BLOOM's (2022, 250,000 tokens, deliberately trained on many languages, orange).

How to read it

With the GPT-4 tokenizer, the same text needs 1.6 times more tokens in French, 4.8 times more in Hindi, and 15 times more in Shan, a language of Myanmar. With GPT-2, the gaps are even larger. BLOOM brings Chinese, Arabic or Hindi close to English, but Burmese and Shan remain above 10 times.

Why

The tokenizer is learned on a given corpus. A language that is well represented in that corpus gets many dedicated tokens, sometimes whole words. A rare language is split into small pieces, sometimes byte by byte.

The consequences (text on the right)

More tokens for the same meaning means more computation, a higher cost per request when paying per token, slower answers and less text in the context window. The authors speak of unfairness between languages.

The data

Petrov et al., "Language Model Tokenizers Introduce Unfairness Between Languages", NeurIPS 2023 (arxiv.org/pdf/2305.15425). They tokenized the FLORES-200 dataset, the same sentences translated into 200 languages, with 17 tokenizers. The figure shown is the ratio between the total number of tokens in the language and in English. More recent tokenizers, such as Qwen3's, are not in the study: the gaps are generally smaller, but the principle remains the same.

Embeddings

Embedding : a learned dense vector for each token. The embedding matrix E has one row per vocabulary token: V × d.



- king
- queen
- riots
- revolution
- table
- chair
- cable



- 0.2, 1.5, …
- 0.2, -1.5, …
- 1.3, -0.1, …
- -0.75, 0.05, …
- -0.3, 0.2, …
- 1.5, 0.5, …
- 0, 0, …

What the slide shows

The factory image: words go in on the left (king, queen, otherwise, mercenary) and come out on the right as lists of numbers. Each word becomes a dense vector, i.e. a point in a multi-dimensional space. The values shown are made up and truncated ("…").

The definition

An embedding is a dense vector learned for each token. Concretely, it is a matrix E of size V × d: one row per vocabulary token (V rows) and d columns. Turning a token into a vector means reading the row that matches its ID, which is mathematically equivalent to multiplying its one-hot vector by E.

The size d

d is an architecture choice fixed before training. It is the width of the vectors throughout the model: about 300 for word2vec (2013), 512 for the 2017 Transformer, 768 for BERT-base, 4,096 for Qwen3-8B. The larger d is, the more information each token can carry, but the heavier the model.

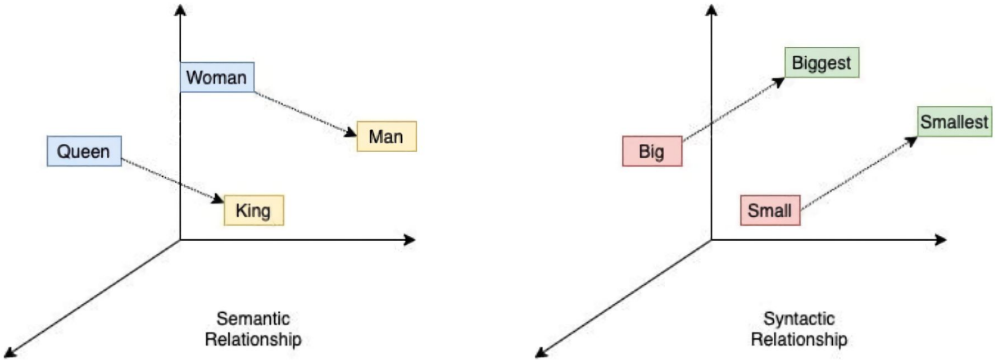
The Qwen3-8B box

151,936 rows × 4,096 columns ≈ 622 million parameters for this table alone, i.e. about 1.2 GB in BF16.

How the values are learned

At the start, the matrix is filled with random numbers. During training, after each batch of text, backpropagation computes how to change each parameter to reduce the prediction error, including the rows of the tokens present in the batch. The optimizer moves them slightly, and this is repeated over billions of sentences. When the model is used, the matrix no longer changes.

Embedding capture relationship between words



What the slide shows

Two embedding spaces drawn in 3D. On the left, a semantic relation (related to meaning): the Woman → Man arrow is parallel to the Queen → King arrow; this is the "gender" direction. On the right, a syntactic relation (related to grammar): Big → Biggest is parallel to Small → Smallest; this is the superlative direction. At the bottom, the famous arithmetic: king – man + woman ≈ queen.

Why these directions appear

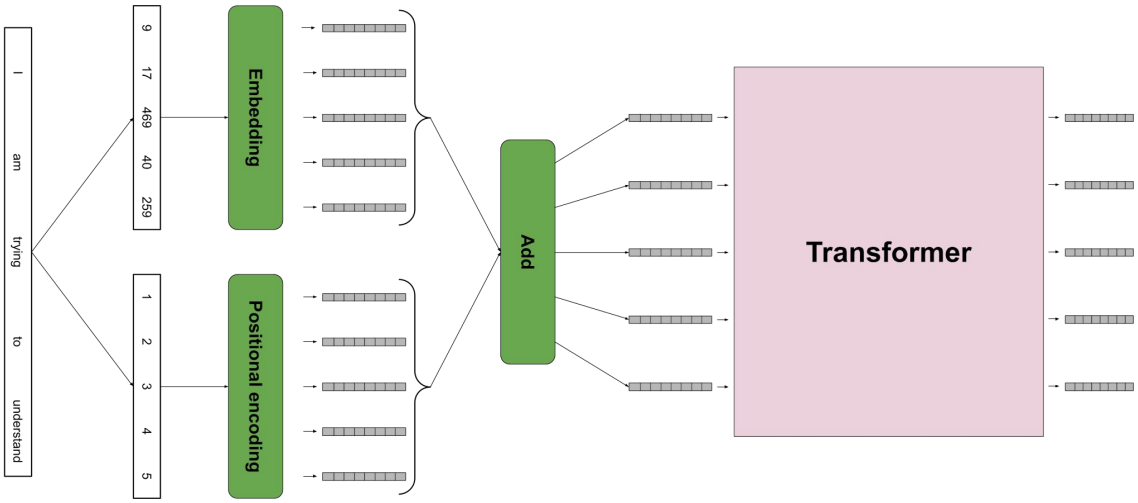
Nobody programs them: they emerge from training. "king" and "queen" appear in almost identical contexts ("the … reigned") and differ only by a few gender-related words ("he / she"). "man" and "woman" differ by exactly the same words. To predict well, the model encodes this difference the same way in both pairs, hence by the same direction. This is the distributional hypothesis: words used in similar contexts get similar vectors (Firth, 1957). These analogies were brought to light with word2vec (Mikolov et al., 2013).

Nuances

- It is approximate: king – man + woman is not exactly "queen", it is its nearest neighbor, excluding the starting words.
- It works well for some relations (gender, country and capital, plural, verb tense) and less well for others.
- Biases in the texts also become directions (for example stereotyped associations between jobs and gender).
- Real spaces have hundreds or thousands of dimensions; 3D is only there for readability.

Pre-processing in NLP with Transformers

Model input : one vector per token, its embedding (what) and its positional encoding (where)



What the slide shows

Everything that happens before the Transformer, from left to right, on the sentence "I am trying to understand":

- the sentence is split into 5 tokens;
- each token is replaced by its ID in the vocabulary (9, 17, 469, 40, 259);
- the "Embedding" block turns each ID into a vector: it says what the word is;
- in parallel, the "Positional encoding" block produces a vector for each position (1 to 5): it says where the word is;
- the "Add" block adds the two vectors, coordinate by coordinate;
- the result, one vector per token, enters the Transformer.

Why add the position

The Transformer processes all tokens at the same time, with no notion of order. Without position information, "the dog bites the man" and "the man bites the dog" would be seen the same way.

Why add (rather than concatenate)

Addition keeps the same size d: a single vector per token carries both the word and its place. There is no extra parameter, and nothing changes in the rest of the model. Example with d = 4: embedding of "trying" (0.20; –0.50; 0.10; 0.70) + vector of position 3 (0.14; –0.99; 0.03; 1.00) = (0.34; –1.49; 0.13; 1.70). The same word at two different places gives two different vectors.

A nuance

The tokenizer is a tool outside the model, and frozen. The embedding, on the other hand, is the first layer of the model: its weights are learned with everything else.

Today

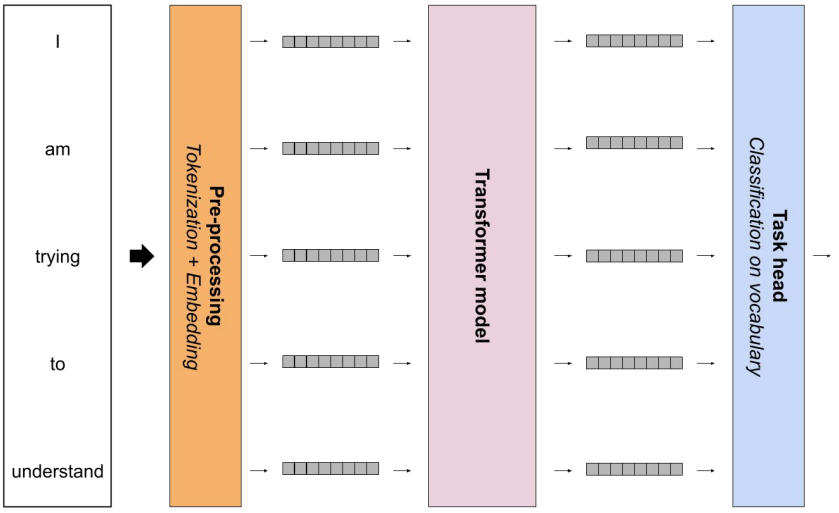
This is the 2017 Transformer diagram. In Qwen3-8B, no position vector is added to the input anymore: position is injected directly into attention (RoPE, attention part).



Transformers fundamentals

cc

Example of NLP system



What the slide shows

A complete NLP system, from left to right, on the sentence "I am trying to understand":

- the sentence split into tokens (white box);
- pre-processing (orange): tokenization and embedding, which give one vector per token;
- the Transformer model (pink), which transforms these vectors;
- the task head (blue), here a classification over the vocabulary;
- the output: the predicted token, "Transformers".

What to notice

5 vectors go into the Transformer and 5 come out, of the same size. The Transformer does not change the shape of the data: it enriches each vector with information from the other tokens. Here the task head takes the last vector and turns it into a score for each vocabulary token, then into probabilities with a softmax. The most likely token is the next word: this is exactly what a language model does.

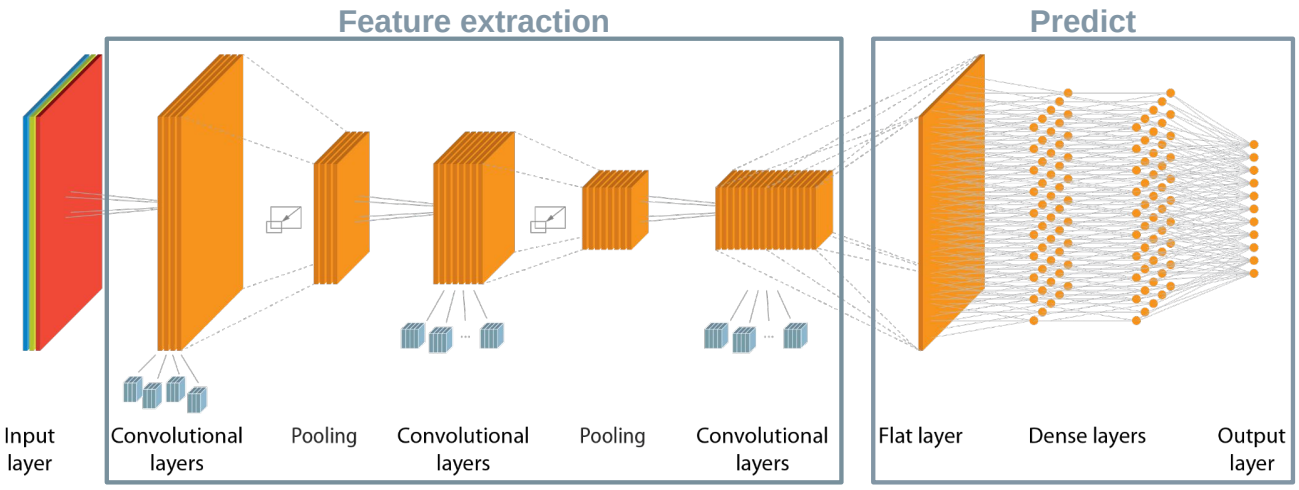
The task head

It is usually a simple linear layer suited to the problem: predicting the next word (one score per vocabulary token), classifying a sentence (positive or negative), labeling each word (person name, place…). The same Transformer can be kept and only the head changed for another task: this idea comes back with fine-tuning.

What is learned

The embedding, the Transformer layers and the head are learned parameters. The tokenizer, on the other hand, is frozen before training. The drawing places the embedding in pre-processing for readability, but it really is the first layer of the model.

Analogy with vision: a CNN image classifier



What the slide shows

A convolutional network (CNN) for image classification, presented as an analogy, since many students already know this kind of network. It is split into two parts:

- "Feature extraction": convolution and pooling layers turn the input image (the red rectangle) into increasingly abstract features;
- "Predict": flattening (flat layer) then dense layers produce the image class (output layer).

The CNN building blocks

- Convolution: small filters slide over the image and detect local patterns. The first layers detect edges and textures, the next ones shapes, then objects.
- Pooling: reduces the size of the feature maps while keeping the essentials.
- Dense layers: each neuron is connected to all those of the previous layer; they combine the features to decide.

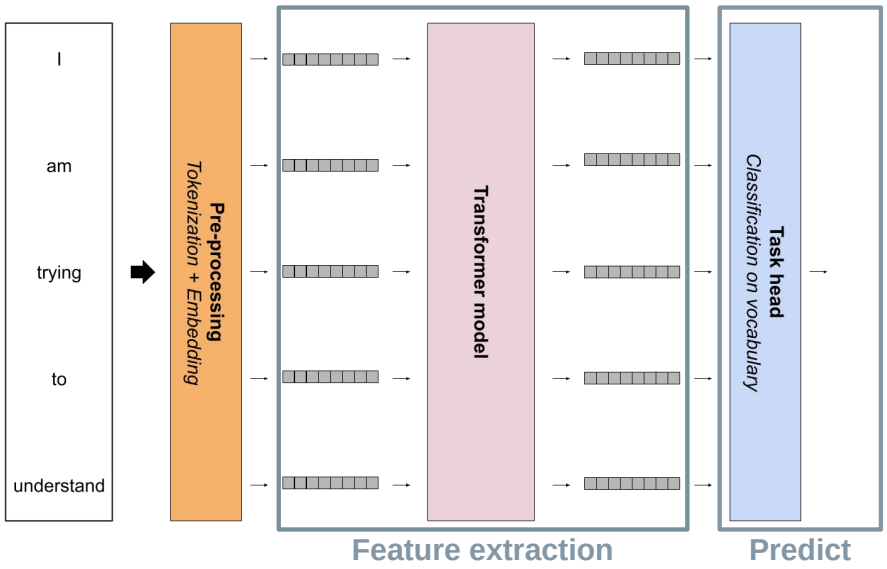
What to take from the analogy

The split into two roles: first extract a good representation of the data, then use it for a specific task. The next slide applies this split to the NLP system.

Limit of the analogy

There is nothing convolutional in the Transformer. A convolution only looks at a local neighborhood, whereas attention directly connects each token to all the others, whatever their distance.

Feature extraction with Transformers



What the slide shows

The same NLP system as two slides earlier, framed according to the CNN split:

- "Feature extraction" groups the pre-processing and the Transformer model;
- "Predict" corresponds to the task head.

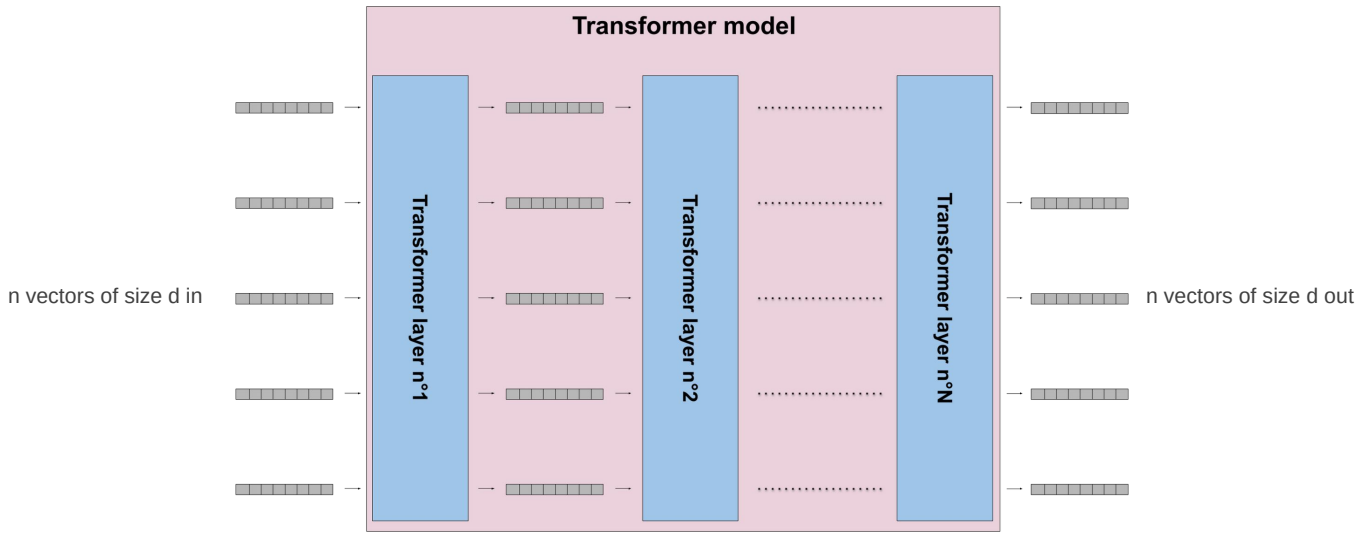
What feature extraction produces

Contextual representations: the output vector of "understand" no longer represents only the isolated word, but the word in the context of the whole sentence. This is the key difference with the initial embedding. The embedding of a word is always the same, whatever the sentence. After the Transformer, the vector depends on the context: "bank" will not have the same vector in "river bank" and in "bank account". It is the attention mechanism that produces this effect, covered in the next part.

Why this split is useful

The "extraction" part is the most expensive to train and the most general: it learns the language. The "prediction" part is small and task-specific. Extraction can therefore be trained once on huge amounts of text, then adapted to many tasks at low cost. This is the principle of pre-training followed by fine-tuning, presented in the last part of the course.

Vanilla Transformer architecture



What the slide shows

The inside of the pink "Transformer model" box: a stack of Transformer layers ("Transformer layer n°1, n°2… n°N"). On the left, n vectors of size d go in, one per token; on the right, n vectors of the same size come out. Between two layers, the arrows show that the output of each layer is the input of the next.

Key points

- The N layers have the same structure, but each one has its own weights.
- Each layer keeps the shape of the data (n vectors of size d), which makes it possible to stack as many as needed. The depth N is one of the main levers of a model's size.
- A frequently cited intuition: the first layers capture rather local and grammatical information, the deep layers more abstract and semantic information.

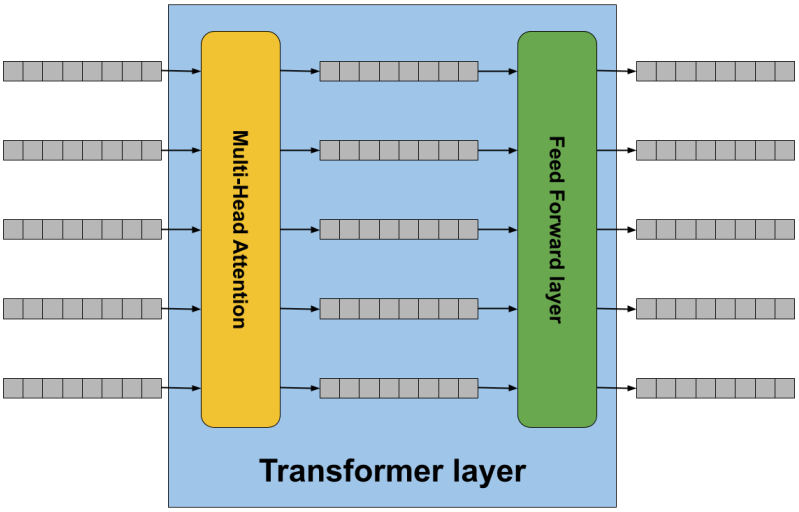
Orders of magnitude

The original Transformer (2017) had 6 layers in the encoder and 6 in the decoder, with d = 512. Today's large models stack dozens of layers with much wider vectors.

The Qwen3-8B box

36 layers and vectors of size d = 4,096. Each Qwen3-8B layer has about 193 million parameters; the 36 layers account for about 7 billion, out of 8.2 billion in total. The rest is in the embedding matrix and the output head.

Transformer layer simplified



What the slide shows

A simplified Transformer layer, reduced to its two blocks. The 5 input vectors first go through the "Multi-Head Attention" block (yellow), then through the "Feed Forward layer" block (green), and come out as 5 vectors.

The role of each block

- Multi-head attention moves information between tokens: it is the only place in the model where a token "sees" the others. The large yellow rectangle, crossed by all the lines, conveys this sharing.
- The feed-forward processes each token separately, independently of the others. It transforms the information inside each vector.

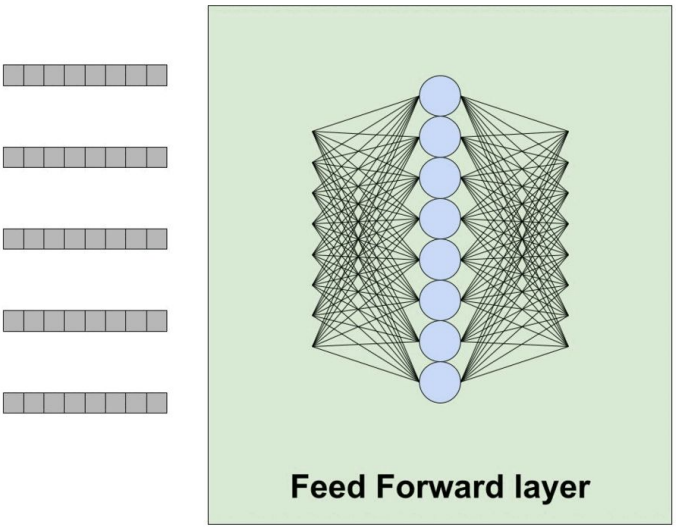
An image to remember

A layer is like a meeting followed by individual work: first each word gathers from the others the information that is useful to it (attention), then it "digests" on its own what it received (feed-forward).

What is missing from this diagram

It is deliberately simplified. The real layer adds after each block a residual connection and a normalization ("Add and normalize"), presented two slides later. The content of the attention block is detailed throughout the next part.

Feed Forward layer



What the slide shows

The feed-forward block: a small classic neural network (multilayer perceptron). The token vectors, on the left, go one by one through the same network, shown in the center. The animation shows this passage token after token.

How it is built

A first linear layer widens the vector, a non-linear activation function is applied, then a second linear layer brings it back to its original size. In the original Transformer, the hidden layer is 4 times wider than the vector (512 $\rightarrow$ 2,048 $\rightarrow$ 512), with the ReLU activation.

The key point

There is not one network per position: it is a single network, with the same weights, applied to each token separately. The original paper describes it as a network applied "to each position separately and identically". In practice, all tokens are processed at the same time as a matrix product, which is very efficient on GPUs. Since it processes each token in isolation, this block cannot move information between words: that is the role of attention.

Its weight in the model

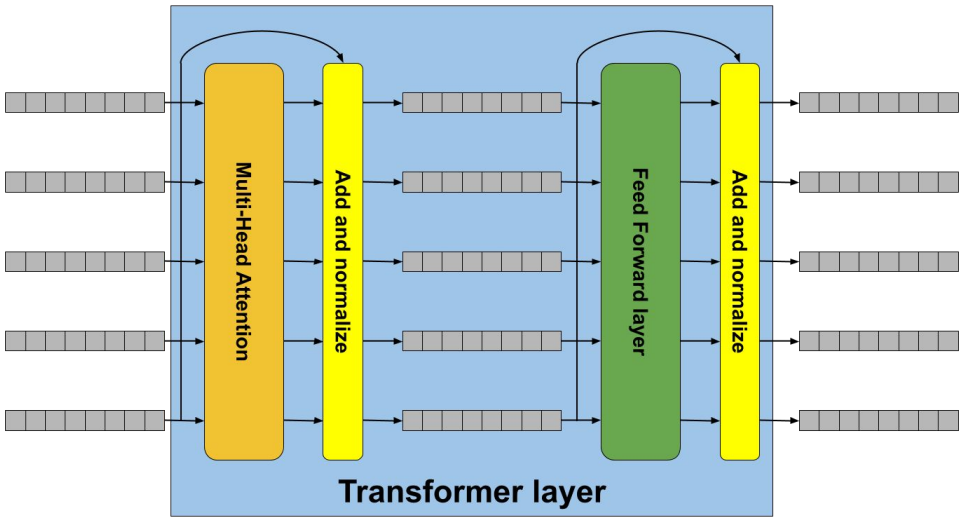
The feed-forward holds most of the parameters. It is often interpreted as a "memory" where the model stores a large part of its knowledge.

The Qwen3-8B box

Qwen3-8B uses a "gated" variant, SwiGLU, with three matrices instead of two. The vector is projected twice to 12,288 dimensions; one of the projections goes through an activation (SiLU) and acts as a "gate" that multiplies the other element-wise; the result is brought back to 4,096. That is about 151 million parameters per layer, i.e. three quarters of the layer.

Transformer layer

Residual connection : each sub-layer adds a correction to its input, a normalization then rescales the vector.



What the slide shows

The complete Transformer layer. Compared with the simplified version, a yellow "Add and normalize" block follows each sub-block (attention, then feed-forward). The curved arrows at the top bypass each sub-block: these are the residual connections.

The residual connection ("Add")

The input of the sub-block is added to its output: x + f(x). The sub-block therefore does not have to produce a complete representation, it only learns a correction to add. The original information flows directly from one end of the model to the other and each layer contributes to it; this is called the "residual stream". Without these connections, the signal and the gradient weaken as they go through dozens of layers, and training becomes very difficult.

Normalization ("normalize")

For each token, the vector coordinates are re-centered and rescaled (LayerNorm: subtract the mean, divide by the standard deviation, then apply a learned scale factor). Values stay in a stable range from one layer to the next.

What these two elements bring

They make it possible to stack many layers and train the whole thing stably.

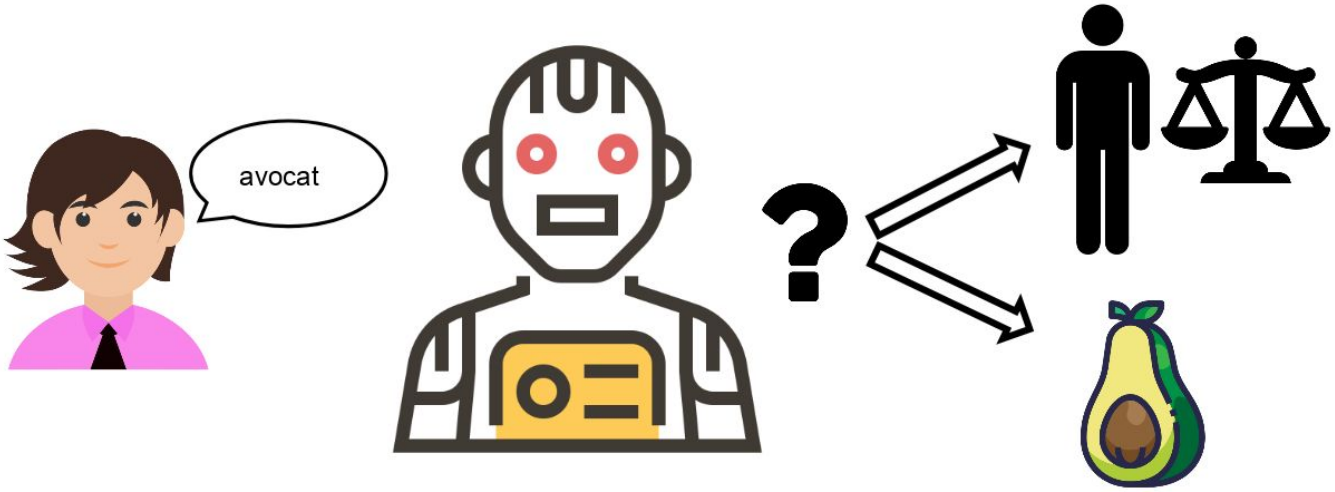
The Qwen3-8B box

The diagram is the 2017 one: normalization comes after the addition (post-norm). Qwen3-8B, like most current LLMs, places it before each sub-block (pre-norm): x + f(Norm(x)). It uses RMSNorm, a simplified version of LayerNorm that does not subtract the mean and only divides by the root mean square. Placing it before each sub-block makes training very deep models more stable.



Attention mechanism

One word, several meanings



What the slide shows

A person says "avocat", and the robot (the model) hesitates between two meanings: avocat the lawyer (the man and the scales) and avocat the fruit (avocado). The example is deliberately in French, because the word has two very different meanings.

The problem

This is polysemy: the same word has several meanings. Yet at the output of the embedding, the vector of "avocat" is always the same, whatever the sentence: it is a fixed row of the embedding matrix. A fixed vector cannot usefully carry both meanings at once. At this stage, the model therefore has no way of knowing which "avocat" is meant.

How a human does it

We look at the surrounding words. The context decides the meaning, and this is exactly what the attention mechanism will do.

Other examples to mention

Polysemy is everywhere: "mouse" (animal or device), "bank" (financial institution or river bank), "operation" (surgical, mathematical, military). Beyond word meaning, a lot of information depends on context: what a pronoun ("he", "she") refers to, what the subject of a verb is, which noun an adjective qualifies.

What comes next

The next slide adds context to the sentence and shows that the choice of meaning becomes obvious.

Context decides

Context : the surrounding words that tell which meaning is meant. Attention lets every token read its context.



What the slide shows

The same situation, but the person says a complete sentence: "J'ai ajouté un avocat frais à ma salade de pâtes" ("I added a fresh avocado to my pasta salad"). The robot thinks of a salad and pasta (the bubble): the "lawyer" meaning is crossed out (red cross), the "fruit" meaning is validated (green check).

Key points

The words "frais" (fresh), "salade" (salad) and "pâtes" (pasta) clearly point to the fruit. To represent "avocat" well, the model must therefore fetch information from the other words of the sentence, giving more importance to the relevant ones ("salade", "frais") than to the others ("J'", "à", "ma").

The definition

The context is the surrounding words that indicate which meaning is intended. Attention is the mechanism that lets each token read its context: it updates each token's vector by mixing in information from the other tokens, with weights that depend on their relevance.

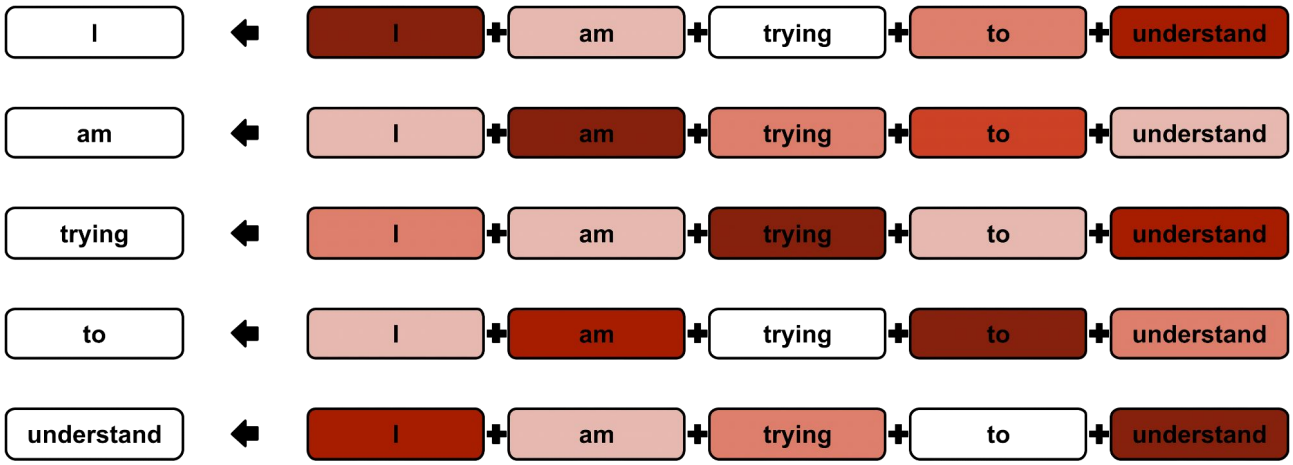
The expected result

After an attention layer, the vector of "avocat" is no longer the same depending on whether it is surrounded by "salade" or "tribunal" (court). We move from a fixed representation (the embedding) to a contextual representation.

Important point

All words do the same thing at the same time: "salade" also looks at "avocat", "frais" looks at "avocat"… Each token updates its own representation from the others.

Intuition behind the attention mechanism



What the slide shows

The idea of attention in one picture, on the sentence "I am trying to understand". On the left, the new representation of each word. On the right, a row of 5 boxes, one per word of the sentence. The color of each box indicates its weight: the darker it is, the more the word counts in the new representation.

How to read it

Each row reads as a weighted sum: new vector of "I" = weight × I + weight × am + weight × trying + weight × to + weight × understand. For example, the new representation of "I" relies mostly on "I" itself and on "understand", and little on "am" or "trying". The row of "to" relies mostly on "am" and "to".

Key points

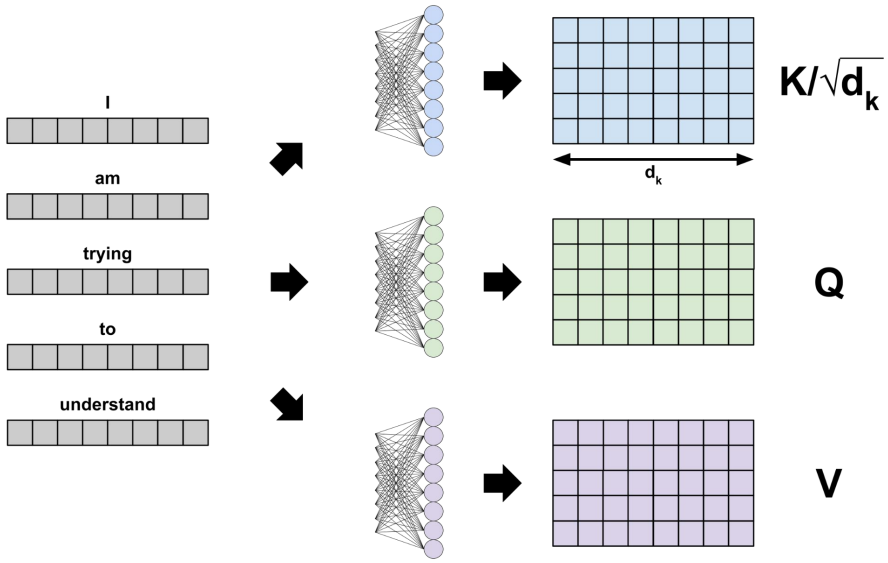
- The new vector of each word is a weighted average of the vectors of all the words in the sentence.
- The weights are positive and sum to 1 on each row.
- A word can rely strongly on itself, on a neighbor or on a distant word: distance in the sentence is not an obstacle, unlike in RNNs.
- The weights are different for each word and each sentence: they are recomputed every time.

The definition

The original paper describes an attention function as mapping a query and a set of key-value pairs to an output (Vaswani et al., 2017). The next three slides show where these queries, keys and values come from, and how the weights are computed.

Attention explained - 1

Query, key, value : three learned projections of each token: what it looks for (Q), what it offers (K), what it passes on (V).



What the slide shows

The first step of the computation. The 5 input vectors (left) go through three different linear layers (the three small networks), which produce three matrices: K (keys, blue), Q (queries, green) and V (values, purple). Each matrix has one row per token. The width of the key and query matrices is written d■; the matrix K is divided by √d■ (explained below).

The three roles

A library analogy:

- the query (Q): what the token is looking for ("where is my subject?");
- the key (K): the label that says what each token can offer ("I am a noun");
- the value (V): the content actually passed on if the token is chosen.

Formally

Q = X·W_Q, K = X·W_K, V = X·W_V, where X is the matrix of input vectors and W_Q, W_K, W_V are weight matrices learned during training. The same weights are applied to all tokens.

Why three projections

A token does not play the same role depending on whether it is looking for information, offering it or passing it on. Separating the three gives the model much more flexibility than using X directly.

Why divide by √d■

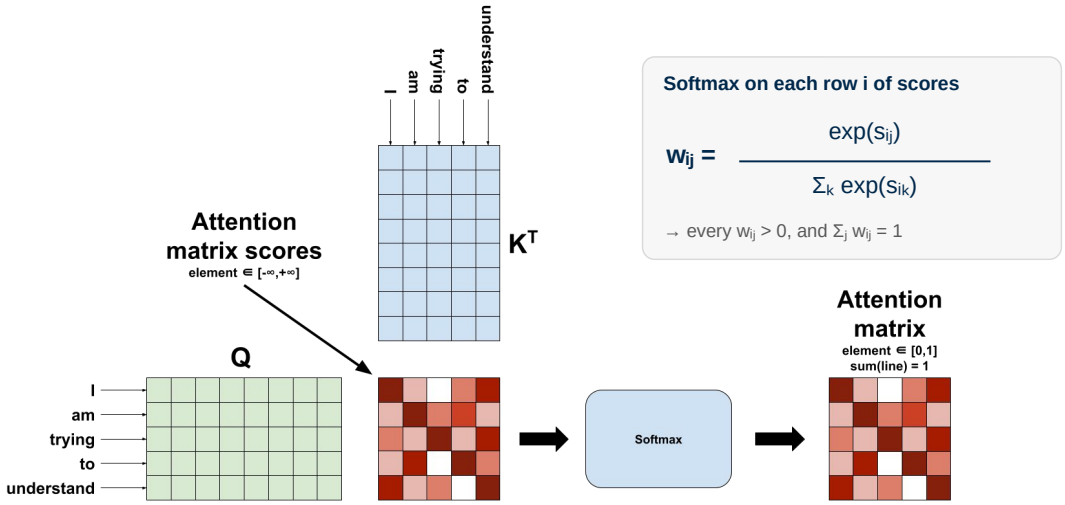
The dot product of two high-dimensional vectors tends to become large. Scores that are too large make the softmax "all or nothing" and the gradients become tiny. Dividing by √d■ brings the scores back to a reasonable scale. The original paper applies this division to the product Q·K■; placing it on K, as here, gives the same result.

What is learned

Only the three weight matrices. Q, K and V are recomputed for each sentence.

Attention explained - 2

Softmax : turns each row of scores into positive weights that sum to 1: the attention matrix.



What the slide shows

The second step. The query matrix Q (green, one row per token) is multiplied by the transpose of the keys K■ (blue, one column per token). This gives a square matrix of scores, 5 × 5 here ("attention matrix scores", arbitrary values between −∞ and +∞). A softmax is then applied row by row, giving the attention matrix (values between 0 and 1, each row sums to 1).

Reading the matrix

Cell (i, j) is the dot product between the query of token i and the key of token j: it measures how well token j answers what token i is looking for. The more the two vectors point in the same direction, the higher the score. Each row of the attention matrix therefore gives how one token's attention is spread over all the others.

The softmax formula (box at the top left)

w■■ = exp(s■■) / Σ■ exp(s■■), where s■■ is the score of cell (i, j) and the sum runs over all tokens k of row i.

- The exponential makes all weights strictly positive, even for negative scores.
- Dividing by the row sum normalizes: the weights of a row sum to 1, which makes it a probability distribution.
- The exponential amplifies differences: the highest score takes the largest share, low scores are squashed toward 0, without ever being exactly 0.

Example: scores (2, 1, 0) give about (0.67; 0.24; 0.09).

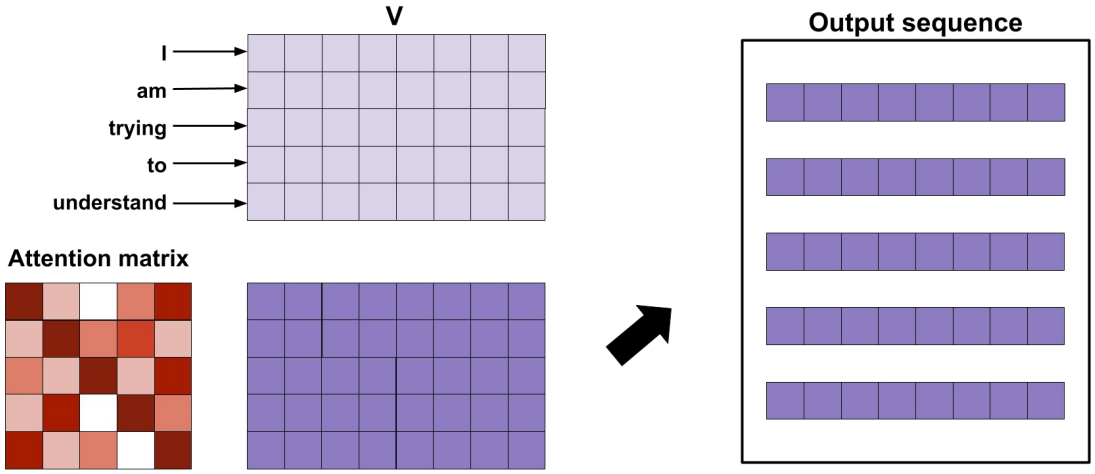
The matrix is not symmetric

Q and K come from different projections: what i looks for in j is not what j looks for in i. In "an artist", "an" brings a lot of information to "artist", while "artist" brings little to "an".

The cost

The matrix has n × n cells for n tokens. Doubling the text length multiplies its size by four: this is the main limitation of attention on very long texts.

Attention explained - 3



What the slide shows

The third step. The attention matrix (red, 5 × 5) is multiplied by the value matrix V (purple, one row per token). The result is the output sequence: one new vector per token.

How to read it

Each row of the result is a combination of the rows of V, with the corresponding row of the attention matrix as coefficients. The new vector of "I" is thus weight(I→I) × V_I + weight(I→am) × V_am + … + weight(I→understand) × V_understand. This is exactly the intuition of the slide "Intuition behind the attention mechanism": a weighted average of the values.

The full formula

Attention(Q, K, V) = softmax(Q·K■ / √d■) · V. This is the most important formula of the course. It boils down to three steps:

- project the inputs into Q, K and V;
- compute the weights: softmax(Q·K■ / √d■);
- mix the values: weights × V.

The sizes

With n tokens: Q, K and V are n × d■, the attention matrix n × n, and the output n × d■. We come out with as many vectors as went in, which then allows the residual connection and the pass through the feed-forward.

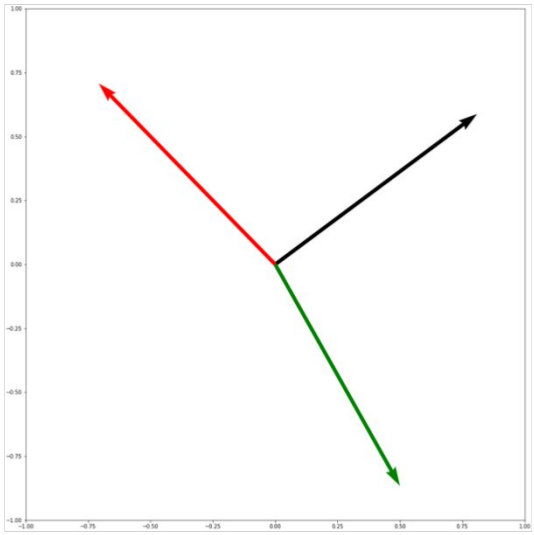
Vocabulary

We speak of self-attention when Q, K and V all come from the same sequence, as here. We will speak of cross-attention when the queries come from one sequence and the keys and values from another (encoder-decoder architecture).

Worked example: “The big dog” (1/3)

The big dog

The : (0.50, -0.87)
big : (-0.70, 0.70)
dog : (0.81, 0.59)



What the slide shows

The start of a toy example, fully worked out by hand over the next three slides. The sentence is "The big dog", with embeddings of only 2 dimensions: The = (0.50; –0.87), big = (–0.70; 0.70), dog = (0.81; 0.59). On the right, these three vectors are drawn as arrows in a plane (green for The, red for big, black for dog).

Why a toy example

In a real model, vectors have hundreds or thousands of dimensions (4,096 for Qwen3-8B), but the computation is exactly the same. With 3 tokens and 2 dimensions, the vectors can be visualized and all computations done on paper. The three vectors have a length close to 1: only their direction matters here.

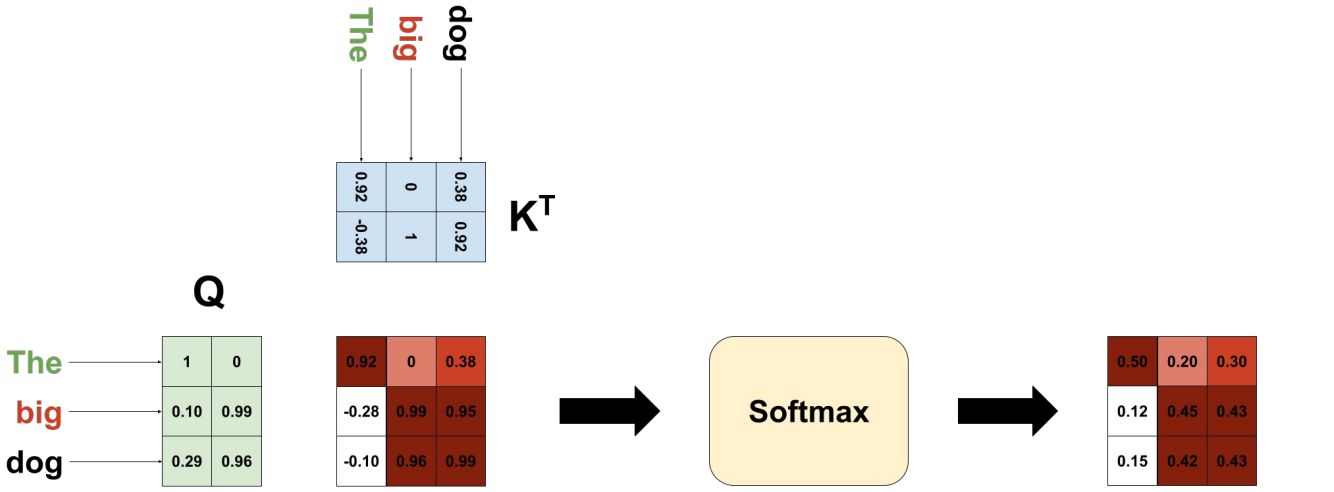
How to read the drawing

Two vectors pointing in similar directions will have a high dot product. The learned Q and K projections will rotate and distort these arrows to decide which words "attract" each other. Here, we want to see the adjective "big" and the noun "dog" look at each other.

The simplifications of the example

- The queries and keys are given directly on the next slide, without detailing the projections.
- The value projection leaves the vectors unchanged: V equals the initial embeddings.
- We do not divide by √d■, to keep the numbers simple.

Worked example: “The big dog” (2/3)



What the slide shows

Computing the attention weights. On the left, the queries Q: The = (1; 0), big = (0.10; 0.99), dog = (0.29; 0.96). At the top, the keys K■, one column per word: The = (0.92; –0.38), big = (0; 1), dog = (0.38; 0.92). In the center, the scores Q·K■; on the right, the attention matrix after the softmax.

Computing one cell

First cell: q_The · k_The = 1 × 0.92 + 0 × (–0.38) = 0.92. The score matrix is:

- The: (0.92; 0; 0.38)
- big: (–0.28; 0.99; 0.95)
- dog: (–0.10; 0.96; 0.99)

Computing the softmax, first row

exp(0.92) ≈ 2.51; exp(0) = 1; exp(0.38) ≈ 1.46; total ≈ 4.97. The weights are therefore 2.51 / 4.97 ≈ 0.50; 1 / 4.97 ≈ 0.20; 1.46 / 4.97 ≈ 0.30. The full attention matrix is:

- The: (0.50; 0.20; 0.30)
- big: (0.12; 0.45; 0.43)
- dog: (0.15; 0.42; 0.43)

How to read it

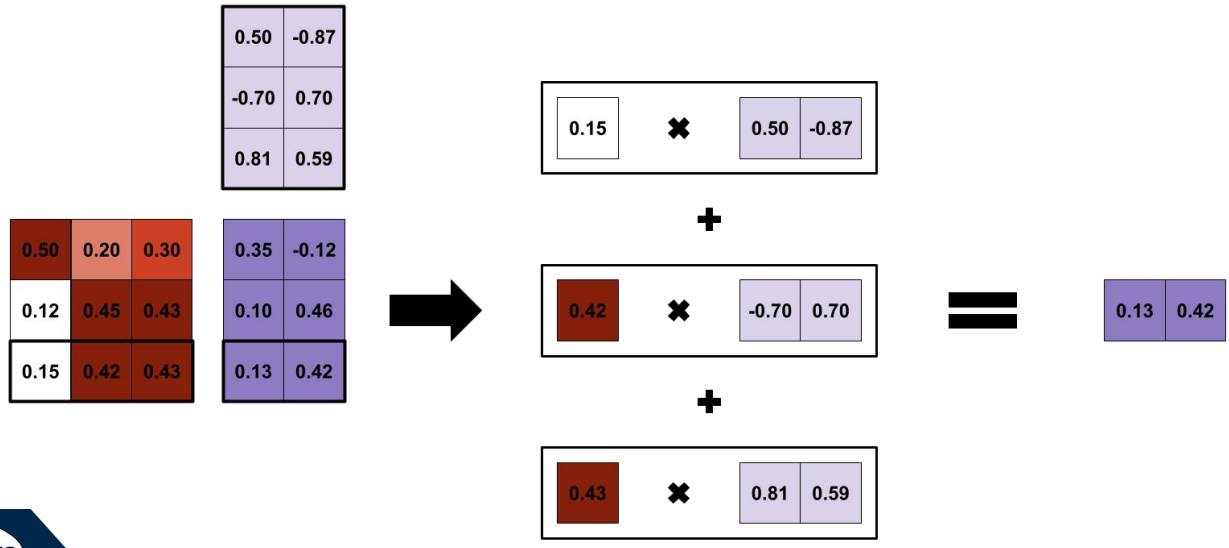
"big" and "dog" look at each other a lot and little at "The": the learned projections have brought the adjective and the noun together. Each row does sum to 1.

Exercise

Redo the second row from the scores (–0.28; 0.99; 0.95): you should get about (0.12; 0.45; 0.43).

Worked example: “The big dog” (3/3)

Weighted sum — “dog” becomes 0.15 × The + 0.42 × big + 0.43 × dog: its vector now carries “big”.



What the slide shows

The last step of the example: the attention matrix (red) multiplied by V (purple, here equal to the initial embeddings) gives the new vectors (dark purple). On the right, the detail of the last row, the one for "dog".

The computation for "dog"

0.15 × The + 0.42 × big + 0.43 × dog = 0.15 × (0.50; –0.87) + 0.42 × (–0.70; 0.70) + 0.43 × (0.81; 0.59) = (0.13; 0.42).

The three new vectors

- The: (0.35; –0.12)
- big: (0.10; 0.46)
- dog: (0.13; 0.42)

Interpretation

The new vector of "dog" contains almost as much "big" as "dog": it now represents "a dog that is big". Compared with the initial vector (0.81; 0.59), it has rotated toward the direction of "big". This is exactly the effect we wanted with the "avocat" example: the word has been enriched by its context.

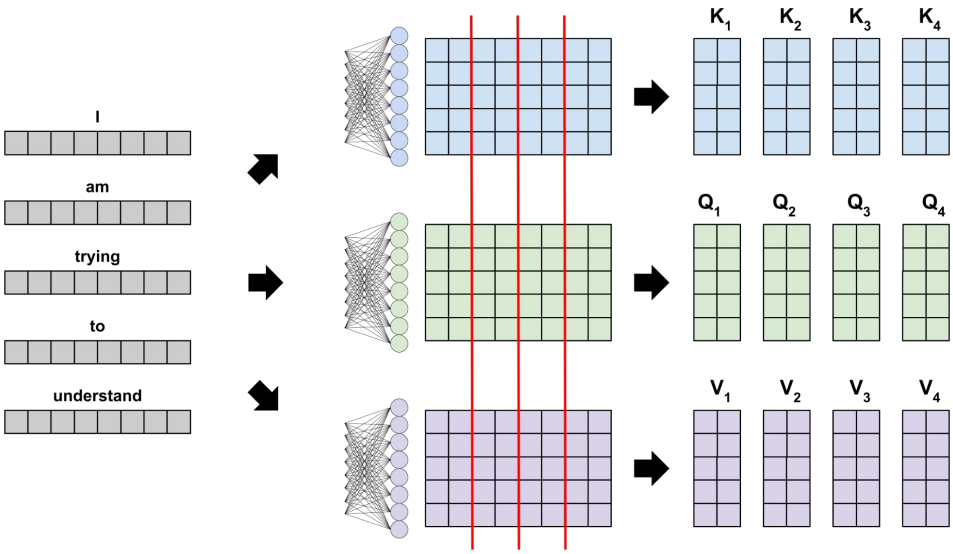
In a real model

The residual connection then adds this result to the original vector: "dog" keeps its identity while integrating the "big" information. Then the feed-forward transforms this vector, and the next layer starts again with new Q, K and V projections.

What is learned, what is computed

Only the projection matrices are learned parameters. The attention matrix is never stored: it is recomputed for each sentence, which lets the same model understand the same word differently in two sentences.

Multi-head attention - 1



What the slide shows

Multi-head attention, first step. As before, the 5 vectors go through the three projections and give the matrices Q, K and V. The red lines then cut each matrix along the columns into 4 pieces: K■ to K■, Q■ to Q■, V■ to V■. Each piece corresponds to one attention head.

The idea

Instead of a single large attention, several are computed in parallel, each on a narrower slice of the vectors. Each head therefore has its own way of comparing tokens and can specialize in one type of relation: one head may link an adjective to its noun, another a pronoun to what it refers to, another simply look at the previous word. These specializations are not imposed: they emerge from training.

The dimensions

If d is the vector size and h the number of heads, each head works on d■ = d / h dimensions. In the original Transformer: 512 / 8 heads = 64 dimensions per head. The split costs almost no extra computation compared with a single attention of size d.

The definition

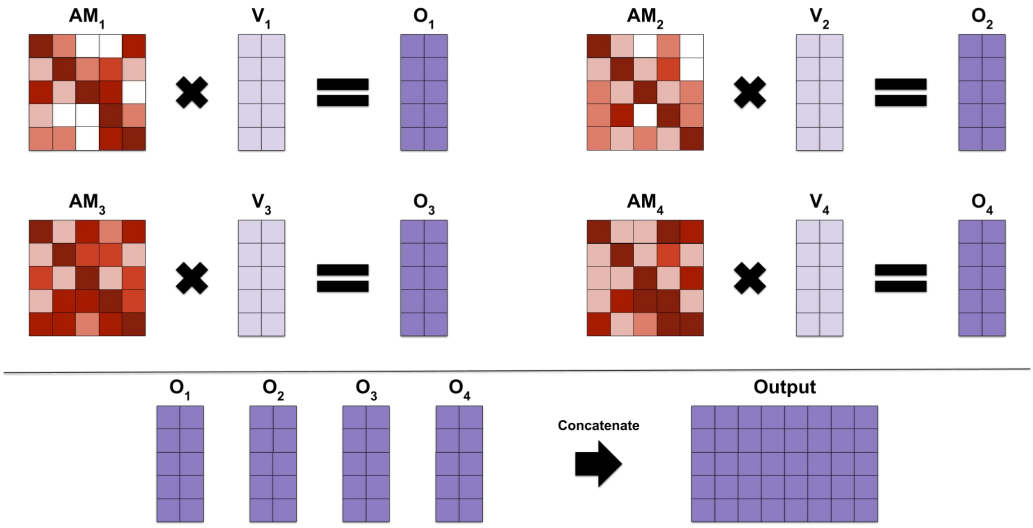
Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions (Vaswani et al., 2017).

The Qwen3-8B box

32 query heads of 128 dimensions each: 32 × 128 = 4,096, the model's vector size. For the keys and values, Qwen3-8B has only 8 heads of 128 dimensions: this is grouped-query attention, two slides later.

Multi-head attention - 2

Concatenation : the h head outputs are put side by side, then mixed by a final linear layer W_O.



What the slide shows

Multi-head attention, continued. At the top, each head i computes its own attention matrix AM■ from Q■ and K■, then multiplies it by its values V■ to get its output O■: AM■ × V■ = O■, and so on for the 4 heads. The patterns of the matrices AM■ to AM■ are different: each head looks at the sentence in its own way. At the bottom, the outputs O■ to O■ are placed side by side ("Concatenate") to rebuild one vector of the original size per token ("Output").

The concatenation

The h outputs, of d■ columns each, are put side by side: we get back h × d■ = d columns. A final linear layer, written W_O, is then applied to mix the information from the different heads. It is not drawn on the slide but is part of the block.

The formula

MultiHead(X) = Concat(head■, …, head■) · W_O, with head■ = Attention(X·W_Q■, X·W_K■, X·W_V■).

Why several heads

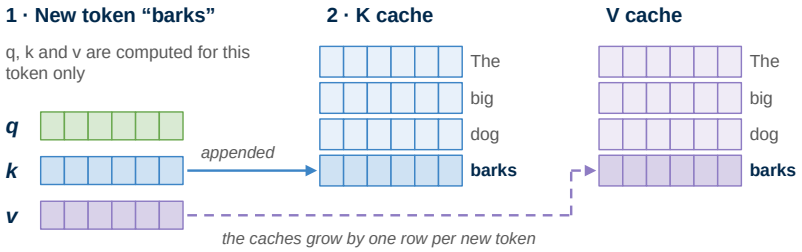
With a single head, each token would have only one attention distribution and would have to compromise between several relations. With several heads, it can look at several things at once: grammar in one head, meaning in another.

The output

It has the same shape as the input: n vectors of size d. It can therefore be added to the input (residual connection) and then passed to the feed-forward.

What is the KV cache?

KV cache : during generation, the keys and values of the tokens already processed are kept in GPU memory, so each new token only computes its own.



3 · Attention of the new token over all cached tokens



Fewer KV heads = a smaller cache: that is grouped-query attention (next slide).

Without the cache, the keys and values of all previous tokens would be recomputed at every new token.

What the slide shows

What happens during generation, when the model produces a new token: here "barks", after "The big dog". The slide prepares the next slide on GQA.

The definition

The KV cache is the memory where, during generation, the keys (K) and values (V) of all tokens already processed are kept. Each new token only computes its own q, k and v.

1 · The new token

The vector of "barks" goes through the three projections and gives q (green), k (blue) and v (purple). This computation concerns this single token only.

2 · The K and V caches

- The K cache contains one row per token already seen ("The", "big", "dog"). The key k of "barks" is appended to it as a new row (highlighted).
- Same thing for the value v in the V cache.
- With each generated token, both caches grow by one row.

3 · The attention of the new token

Only the query of the new token is needed: q · K cache■ gives 4 scores, one per token of the sequence. The softmax turns them into 4 weights (the darkest box is the most useful token), then the weighted sum of the rows of the V cache gives the new vector of "barks". This is the attention seen earlier, reduced to a single row of the matrix.

Why this is possible

During generation, attention is causal: old tokens do not see new ones, so their keys and values never change. They can be computed once and reused. Without the cache, the keys and values of the whole sequence would have to be recomputed for every new token, and the cost of generation would grow very quickly with length.

The price: memory (right)

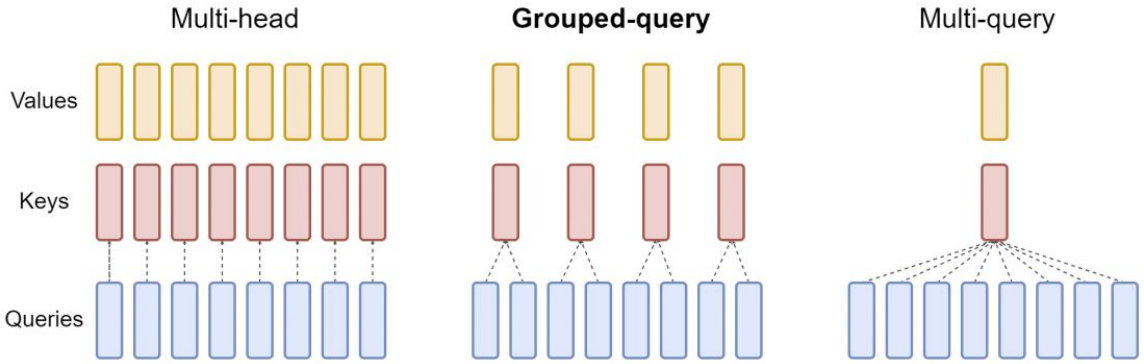
Cache size = 2 (K and V) × number of layers × number of KV heads × head size × bytes per value × number of tokens. There is one cache per layer, and it grows with the context length and the number of requests served at the same time.

The Qwen3-8B box

2 × 36 layers × 8 KV heads × 128 × 2 bytes (bf16) = 147,456 bytes, about 144 KB per token. For a 32,000-token context, that is about 4.8 GB per sequence, on top of the 16 GB of weights. With 32 KV heads (one per query head), it would be 4 times more, about 19 GB.

Sharing keys and values: grouped-query attention

Grouped-query attention (GQA) : several query heads share one key/value head, which shrinks the memory needed to store K and V.



KV cache: during generation, the keys and values of all previous tokens are kept in GPU memory.

Figure 2 of Ainslie et al., EMNLP 2023 github.com/fkodom/grouped-query-attention-pytorch.

What the slide shows

Figure 2 of the grouped-query attention paper (Ainslie et al., 2023), which compares three ways of organizing the heads. At the bottom, the queries (blue), in the middle the keys (red), at the top the values (orange); the dotted lines show which queries use which keys and values.

- Multi-head (left): 8 query heads, and as many key and value heads. This is the attention of the original Transformer.
- Grouped-query (center): the 8 query heads are grouped by 2, and each group shares the same key and value head: 4 K/V heads in total.
- Multi-query (right): a single key and value head, shared by all the queries.

Why share keys and values

To generate text, the model produces one token at a time. To avoid recomputing everything for each new token, it keeps the keys and values of all previous tokens in GPU memory: this is the KV cache. Its size grows with the context length and with the number of K/V heads, and it is often what limits the context length and the number of users served per GPU. Fewer K/V heads means less memory. Multi-query loses a little quality; grouped-query is the compromise, with almost the quality of multi-head and almost the memory of multi-query. Queries, for their part, are never cached.

The Qwen3-8B box

32 query heads for 8 K/V heads (groups of 4). Per token, the cache holds 2 (K and V) × 36 layers × 8 heads × 128 dimensions × 2 bytes ≈ 144 KB. At 32,768 tokens, that is ≈ 4.8 GB, versus ≈ 19 GB with 32 K/V heads.

Source

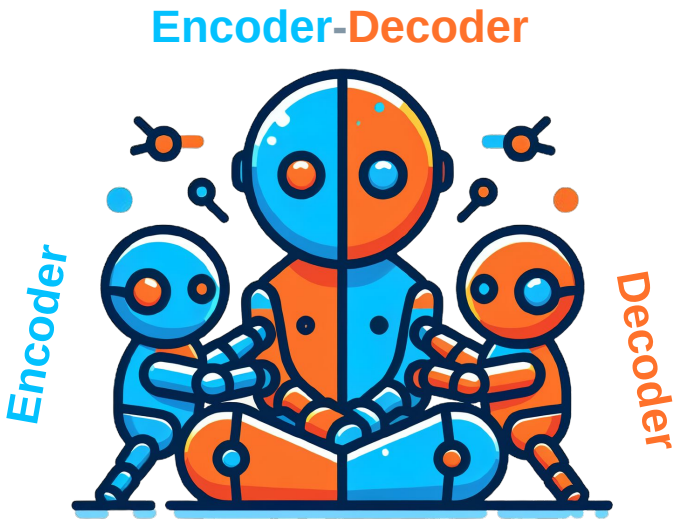
Figure 2 of Ainslie et al., EMNLP 2023 (arXiv:2305.13245), via the repository github.com/fkodom/grouped-query-attention-pytorch.



Transformers architectures

Transformer architectures

Three families : encoder-only (understand), decoder-only (generate), encoder-decoder (turn one sequence into another).



What the slide shows

The three families of architectures built with the building blocks seen so far, represented by three robots: the encoder (blue, left), the decoder (orange, right) and the encoder-decoder (center, half blue, half orange), which combines the two.

The three families

- Encoder only: it reads the whole text at once, each token seeing all the others. It is used to understand: classify a text, extract information, produce a vector that represents a document. Example: BERT (2018).
- Decoder only: it generates text one token at a time, each token seeing only the ones before it. It is used to generate. Examples: GPT, Llama, Qwen3.
- Encoder-decoder: the encoder reads a source sequence, the decoder generates a target sequence by consulting the encoder. It is used to turn one sequence into another: translate, transcribe, summarize. Example: T5 (2019), and the original 2017 Transformer.

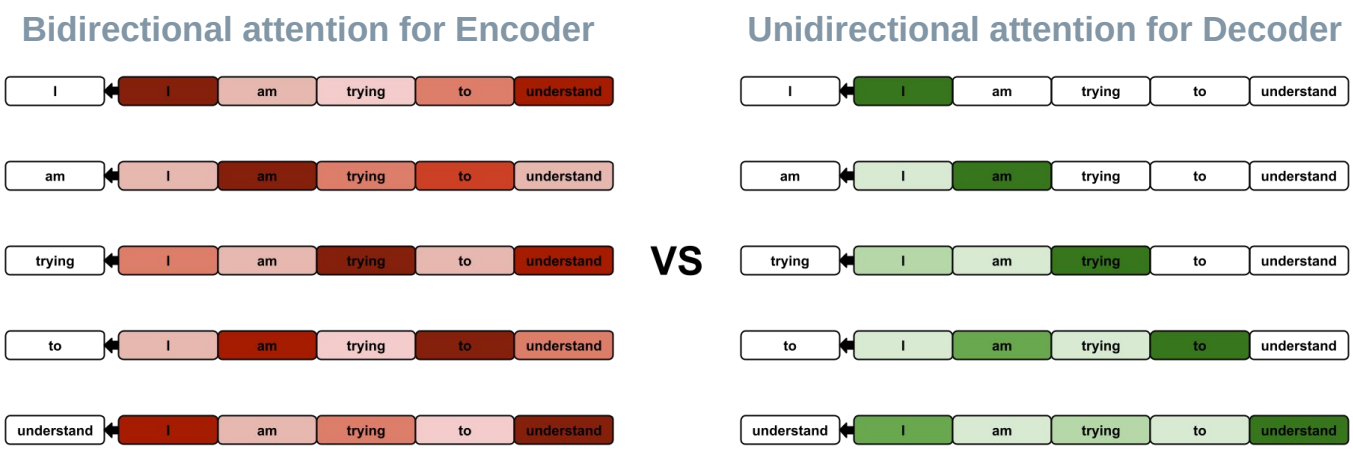
The question that sets them apart

Can each token see the whole sentence, or only what comes before? All the families use the same attention; only the rule of who is allowed to look at whom changes. This is the subject of the next slides.

Today

General-purpose LLMs are almost all decoder-only. Encoders remain widely used for document retrieval and classification. Encoder-decoders remain useful for translation and speech recognition.

Bidirectional vs Unidirectional attention



What the slide shows

The difference between the two types of attention, on the sentence "I am trying to understand". Each row shows which words a token uses to build its new representation; the darker the box, the higher the weight, and a white box has zero weight.

Left: bidirectional attention (encoder)

All the boxes are colored: each word can rely on all the words in the sentence, before and after it. "I" can thus use "understand", even though it is at the end.

Right: unidirectional or causal attention (decoder)

Only the boxes up to the word itself are colored, which forms a triangle. "I" only sees itself; "am" sees "I" and "am"; "understand", the last word, sees the whole sentence. The white boxes correspond to future words, which are forbidden.

Key points

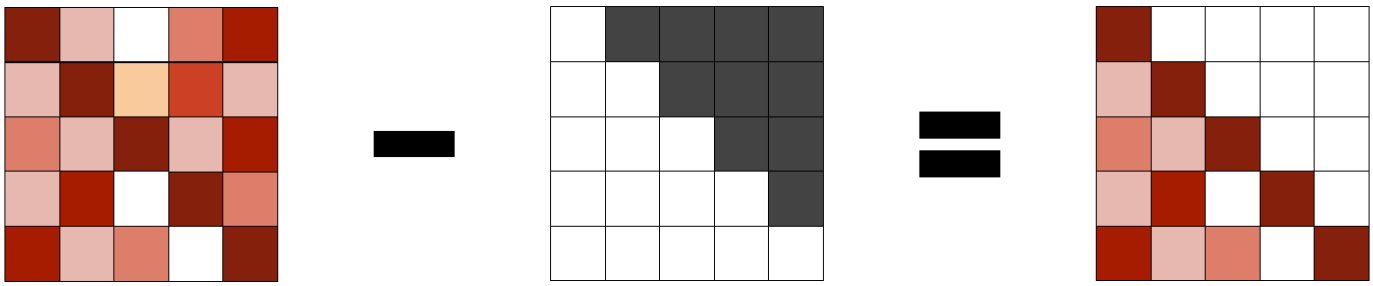
The computation is exactly the same in both cases: Q, K, V, scores, softmax, weighted sum. Only the rule of who is allowed to look at whom changes.

Consequences

- An encoder understands an isolated word better, since it has context on both sides. This is useful to classify or represent a text.
- A decoder is the only one of the two that can naturally generate text from left to right, which makes it the architecture of LLMs.

Mask attention

Causal mask : scores above the diagonal are set to −∞ before the softmax, so future tokens get a weight of exactly 0.



What the slide shows

How causal attention is done in practice. On the left, the full score matrix (5 × 5), as in normal attention. In the center, the mask: the boxes above the diagonal (dark grey) are those where a token would look at a future token. On the right, the result: a triangular matrix where these boxes are empty.

The definition

The causal mask sets the scores above the diagonal to –∞ before the softmax. Since exp(–∞) = 0, these boxes get a weight of exactly zero, and each row still sums to 1 over the remaining boxes.

Why it is very efficient

Thanks to the mask, a whole sentence can be processed in a single pass during training, predicting the next token at every position at the same time, without any position seeing the answer. A sentence of n tokens thus provides n training examples at once. Without the mask, one pass per position would be needed.

During generation

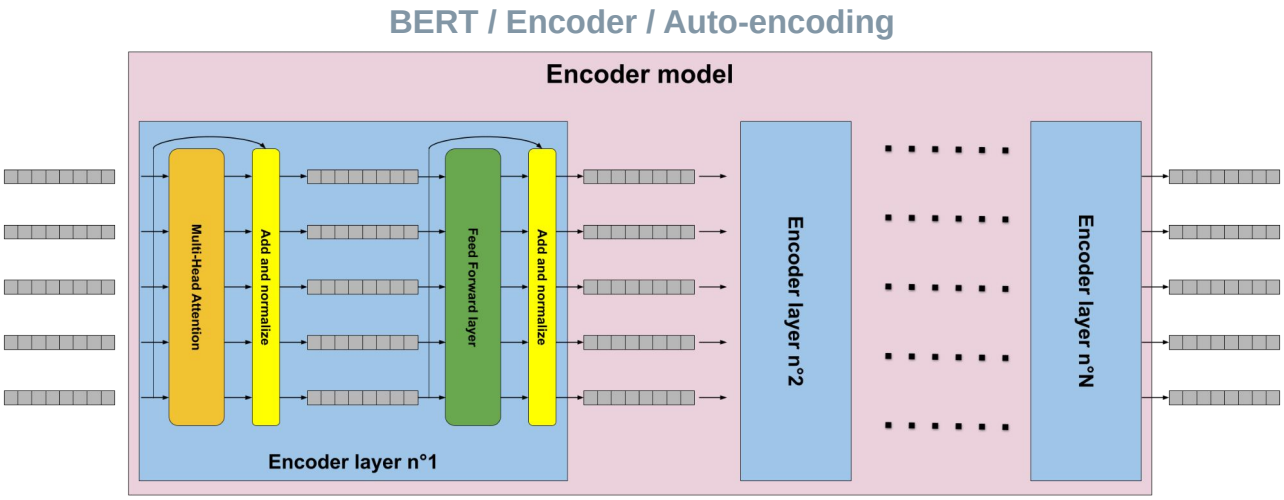
The model adds tokens one by one. Thanks to the KV cache, it only computes the row of the new token, which looks at all the previous tokens: the mask is then implicit.

Note

The mask is the "Mask (opt.)" in the figure from the 2017 paper seen in the attention part: it is optional because it is only used in the decoder.

Encoder architecture

Encoder : bidirectional: every token sees the whole sentence. Still used for classification and for search / RAG embeddings.



What the slide shows

The architecture of an encoder, with the example of BERT ("Bidirectional Encoder Representations from Transformers", Google, Devlin et al., 2019). It is exactly the Transformer seen in the previous part: a stack of N layers, each made of a multi-head attention and a feed-forward, each followed by a residual connection and a normalization ("Add and normalize"). One vector per token goes in and one vector per token comes out.

What characterizes it

Attention is bidirectional: each token sees all the others, both before and after. The output vector of each token therefore takes into account the full context of the sentence.

The term "auto-encoding"

The encoder learns to reconstruct a corrupted input: during BERT's pre-training, about 15 % of the tokens are masked and the model must recover them. This pre-training is presented in the last part.

BERT sizes

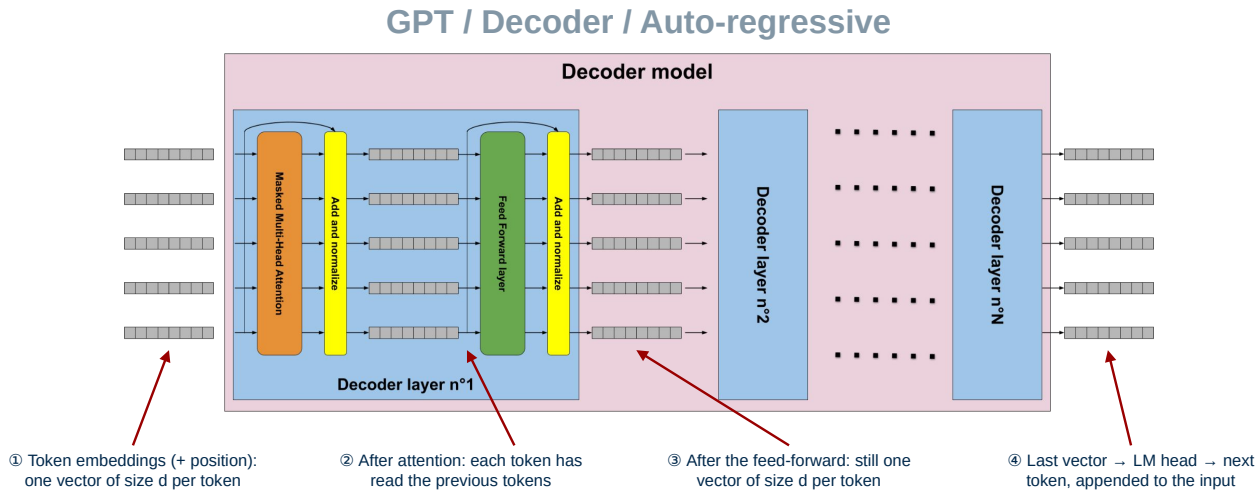
BERT-base: 12 layers, vectors of 768, 110 million parameters. BERT-large: 24 layers, 1,024, 340 million. This is very small compared with today's LLMs.

Current uses

An encoder cannot freely generate text, but it is small, fast and produces very good representations. It remains widely used for classification (sentiment, toxic content detection), entity extraction, and above all to produce document embeddings, the basis of semantic search and of the "retrieval" part of RAG systems. ModernBERT (2024) is a modernized version, with a context of 8,192 tokens.

Decoder architecture

Decoder : causal and autoregressive: it generates one token at a time, each token seeing only the previous ones.



What the slide shows

The architecture of a decoder, with the example of GPT (OpenAI, Radford et al., 2018). The structure is the same as the encoder's, with one difference: the attention block becomes a masked multi-head attention ("Masked Multi-Head Attention", orange).

The state of the tokens at each step (red arrows)

- ① At the input: one vector of size d per token, the token's embedding (with the position information).
- ② After masked attention: still one vector of size d per token, but each vector has integrated the information of the tokens that precede it, and only those.
- ③ After the feed-forward: same shape, each vector has been transformed separately. The output of one layer is the input of the next, up to layer N.
- ④ At the output: only the last vector is used to predict. The output head (LM head) turns it into a score per vocabulary token, the softmax into probabilities, and the chosen token is appended to the end of the input for the next step.

What characterizes it

Attention is causal: each token only sees itself and the tokens that precede it. The mask that achieves this is presented on the next slide.

The term "auto-regressive"

The decoder generates a token, appends it to its input, then generates the next one from the whole sequence, and so on until an end token. Each new token depends on all those already produced.

Why mask

At generation time, the future does not exist yet. The model must therefore learn to predict what comes next only from what precedes. If it saw the next word during training, it would "cheat" and learn nothing useful.

The Qwen3-8B box

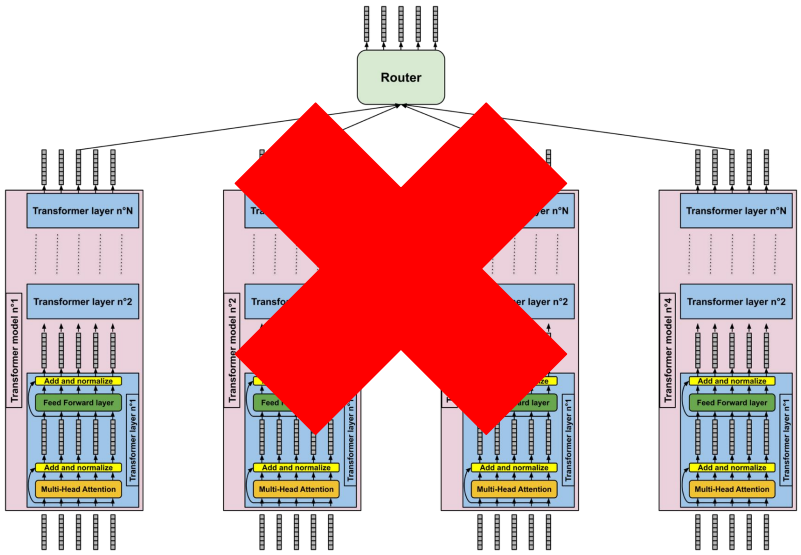
Qwen3-8B is decoder-only, like almost all current general-purpose LLMs (GPT, Llama, Mistral, DeepSeek, Kimi). A single stack of layers is enough to read the question and generate the answer, and this architecture has proven the simplest to scale.

Orders of magnitude

The first GPT had 12 layers and 117 million parameters. Qwen3-8B has 36 layers and 8.2 billion parameters, with the same operating logic.

Misunderstanding about Mixture of Experts

Mixture of Experts (MoE) is not several complete models behind a router: the experts live inside each layer.



What the slide shows

A common misconception about Mixture of Experts (MoE), crossed out in red: several complete Transformer models, each an "expert" in one domain, with a router at the top that would choose which model handles the request.

What it is not

An MoE is not an assembly of models specialized in medicine, law or cooking. There is not one expert per subject that a human could name, and the choice is not made once for the whole request.

The definition

In an MoE, the experts are inside each layer, and the choice is made for each token, at each layer.

The name "expert" is misleading

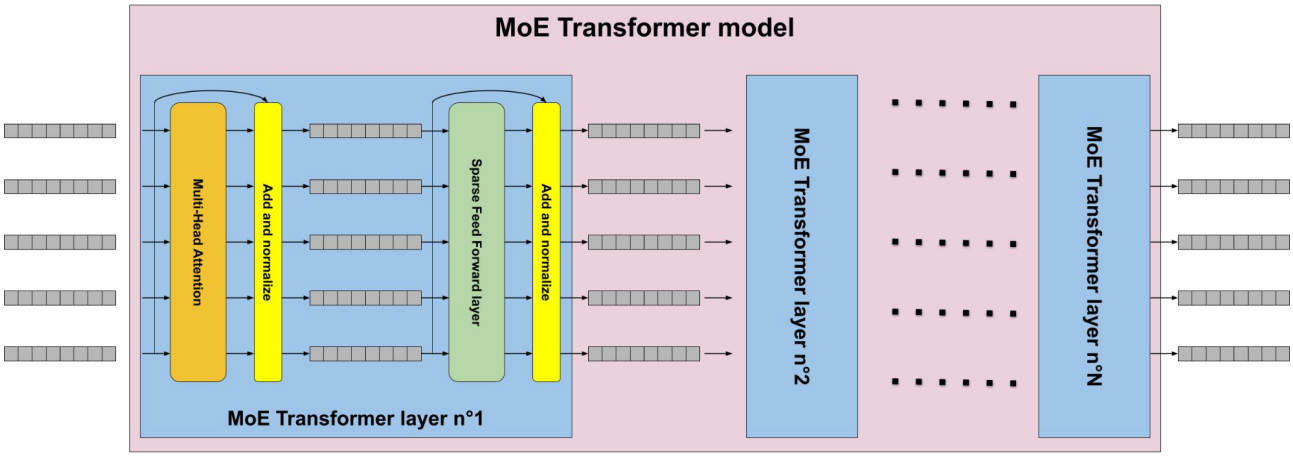
The experts of an MoE are simple feed-forward networks. Their specialization, when there is one, is learned automatically during training and is often hard to interpret: an expert may for example specialize in punctuation, numbers or a certain syntax rather than in a knowledge domain.

What comes next

The next two slides show where the experts really are in the model, then how an MoE layer works.

Mixture of Expert Transformer Architecture

MoE layer the feed-forward block of each layer is replaced by a sparse one, attention is unchanged.



What the slide shows

The real MoE architecture: a classic Transformer, with its N layers, its multi-head attentions (yellow) and its "Add and normalize". The only difference is that the feed-forward block of each layer is replaced by a "Sparse Feed Forward layer" (light green).

The definition

In an MoE layer, the feed-forward is replaced by a sparse block made of several experts; attention does not change.

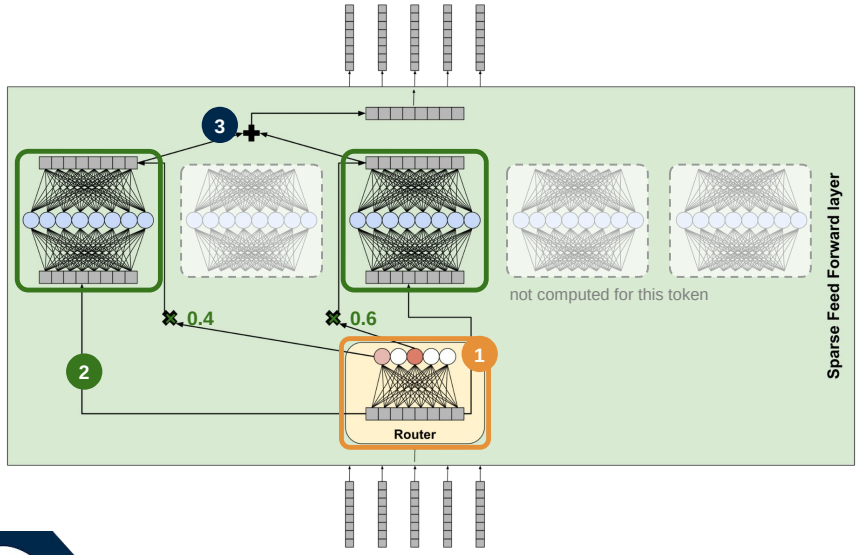
Key points

- The choice of experts is made inside each layer, not once for the whole model.
- Routing is done per token: two tokens of the same sentence can go through different experts in the same layer.
- Routing is done per layer: the same token can go through different experts at layer 1 and at layer 2.

Why the feed-forward

It is the block that holds most of the parameters (two thirds in Qwen3-8B, see the "8.2 billion parameters" slide). By making it sparse, the total number of parameters can be multiplied without multiplying the computation done for each token. Attention, on the other hand, needs to see all tokens and is not split into experts.

Sparse Feed Forward Layer



1 The router scores the 5 experts and keeps the top 2 (darkest circles).

2 The token goes through these 2 experts only (green); the other 3 are skipped (grey).

3 Their outputs are multiplied by the router weights (×) and summed (+).

Each token, in each layer, can pick different experts.

What the slide shows

The detail of a sparse feed-forward layer, with the path of one token highlighted in color. At the bottom, the token vectors come in. In the center, five experts, each a complete feed-forward network. At the bottom center, the router (orange frame), a small network that gives a score to each expert (the colored circles).

The path of one token (badges 1 to 4)

- 1 · The token vector enters the router.
- 2 · The router scores the 5 experts and keeps the 2 best, those with the darkest circles.
- 3 · The token only goes through these 2 experts, framed in green. The other 3, greyed out, are not computed for this token.
- 4 · The outputs of the 2 experts are multiplied by the router weights (here × 0.4 and × 0.6, the crosses), then summed (the "+") to give the output vector, at the top.

The definition

The router is a small network that, for each token, scores the experts and keeps only the top k (here k = 2). The idea goes back to Shazeer et al. (2017).

The choice of experts

The router computes one score per expert, turned into weights by a softmax; the k largest are kept, renormalized to sum to 1. This choice is made again for each token and in each layer: two words of the same sentence can go through different experts, and the same token does not go through the same experts from one layer to the next.

The benefit

The model has many parameters in total, hence a lot of capacity, but each token only uses a small part of them: the computation per token stays close to that of a much smaller model.

The difficulties

- Load balancing: the router must not send all tokens to the same experts, otherwise the others are useless. A term is added to the loss function to encourage a balanced distribution.
- Memory: all experts must be loaded on the GPUs, even those not used for a given token.

A common variant

The shared expert: an expert that is always active for all tokens, in addition to the routed experts. It captures common knowledge and lets the others specialize (DeepSeek-V3, Kimi K3).

Mixture of Experts in today's open models

Active parameters : the parameters actually used for one token. All the experts must still be loaded in memory.

Model	Release	Total	Active / token	Experts used / total
Mixtral 8x7B	Dec 2023	47 B	13 B	2 / 8
DeepSeek-V3	Dec 2024	671 B	37 B	8 / 256
Qwen3-30B-A3B	Apr 2025	30.5 B	**3.3 B**	8 / 128
gpt-oss-120b	Aug 2025	117 B	5.1 B	4 / 128
Kimi K3	2026	2.8 T	104 B	16 / 896

model cards and technical reports (Mixtral, DeepSeek-V3, Qwen3, gpt-oss, Kimi K3 and friends.

What the slide shows

Five large open MoE models, with their release date, total number of parameters, number of active parameters per token and number of experts used out of the total. The row for Qwen3-30B-A3B, the MoE "sibling" of our running example, is highlighted.

The definition

Active parameters are those actually used to compute a token. All the experts must nevertheless be loaded in memory.

Reading the table

- Mixtral 8x7B (Mistral AI): 47 billion parameters, 13 active, 2 experts out of 8.
- DeepSeek-V3: 671 billion, 37 active, 8 routed experts out of 256, plus 1 shared expert.
- Qwen3-30B-A3B: 30.5 billion, 3.3 active, 8 experts out of 128, no shared expert. Its name sums up its two figures: 30 B in total, about 3 B active ("A3B").
- gpt-oss-120b (OpenAI): 117 billion, 5.1 active, 4 experts out of 128.
- Kimi K3 (Moonshot AI): 2,800 billion, 104 active, 16 experts out of 896, plus 2 shared experts.

The two practical consequences

- Computation per token depends on the active parameters: Qwen3-30B-A3B generates about as fast as a dense model with 3 billion parameters.
- GPU memory depends on the total parameters: in BF16, the weights of Qwen3-30B-A3B weigh about 61 GB, those of Kimi K3 about 5.6 TB. Hence the frequent release in reduced precision: gpt-oss-120b, quantized to 4 bits, fits on a single 80 GB GPU.

The trend

More and more, smaller experts (from 8 to nearly 900), with more active experts per token: this is called fine-grained MoE. MoE has become the default architecture of very large open models.

Sources

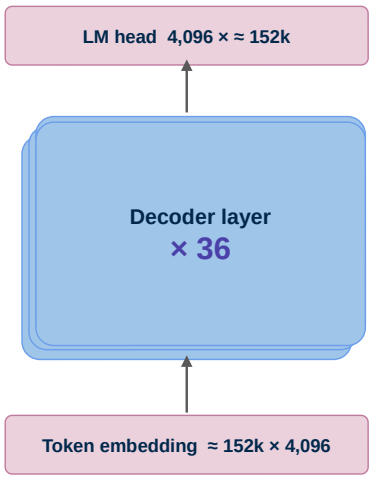
Model cards and technical reports of Mixtral, DeepSeek-V3, Qwen3, gpt-oss and Kimi K3.



LLM today

Qwen3-8B

Qwen3-8B	
Type	Dense, decoder-only Transformer
Parameters	8.2 B
Layers	36
Vector size (d)	4,096
Attention heads	32 query / 8 key-value (GQA)
Vocabulary	151,669 tokens (byte-level BPE)
Context	32k tokens
Training data	36 T tokens, 119 languages



What the slide shows

The ID card of Qwen3-8B, the model used as a real example throughout the course. Each abstract notion (vector size, number of heads, vocabulary…) will be illustrated by its value in this model.

Why this model

It is a "modern classic" dense decoder, with nothing exotic: it illustrates exactly the architecture explained in the course. Its weights are open (Apache 2.0 license) and its design is documented in a detailed technical report. It fits on a single GPU, about 16 GB in BF16.

Reading the table

- Developer: Alibaba's Qwen team. Release: April 2025, with the rest of the Qwen3 family (from 0.6 to 235 billion parameters).
- Type: dense Transformer, decoder only. "Dense" means that all parameters are used for every token, unlike MoEs.
- 8.2 billion parameters, of which about 7 billion are non-embedding.
- 36 identical layers, vectors of size d = 4,096.
- 32 attention heads for the queries but only 8 for the keys and values: this is grouped-query attention (GQA), seen in the attention part.
- Vocabulary of 151,669 tokens, byte-level BPE tokenizer.
- Context: 32,768 tokens natively, 131,072 with the YaRN extension technique.
- Data: about 36 trillion tokens in 119 languages and dialects.

The diagram on the right

The structure of the model: the embedding turns each token into a vector of 4,096 numbers, 36 decoder layers enrich it, and the output head (LM head) turns it into a score for each vocabulary token. The matrices have about 152,000 rows: 151,936 exactly, slightly more than the vocabulary, because the size is rounded for computational efficiency. In this model, the input embedding and the output head are two separate matrices, of about 622 million parameters each.

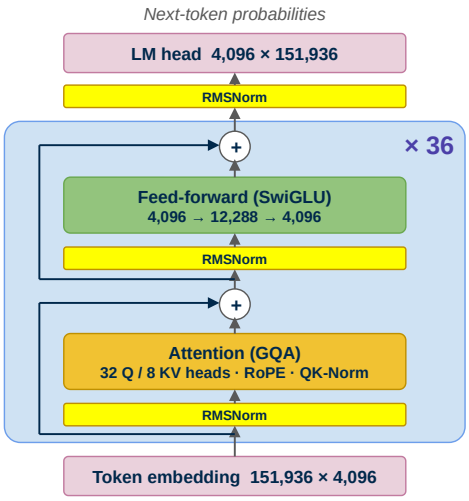
Sources

Qwen3 Technical Report (arXiv:2505.09388), tables 1 and 2; model card and configuration of Qwen/Qwen3-8B on Hugging Face.

From the 2017 Transformer to Qwen3-8B

	Transformer base (2017)	Qwen3-8B (2025)
Architecture	Encoder-decoder	Decoder only
Layers	6 + 6	36
Vector size d	512	4,096
Attention	8 heads (MHA)	32 Q / 8 KV heads (GQA), QK-Norm
Feed-forward	ReLU, 2,048	SwiGLU, 12,288
Normalization	LayerNorm after each sub-layer	RMSNorm before each sub-layer
Position	Sinusoidal, added to the input	RoPE inside attention ($\theta = 10^6$)
Vocabulary	≈ 37k (BPE)	151,669 (byte-level BPE)
Parameters	65 M	8.2 B

Qwen3 Technical Report (arXiv:2505.09388);n.



Qwen3-8B, drawn from its configuration

What the slide shows

On the left, a table comparing the 2017 "base" Transformer with Qwen3-8B, our running example. On the right, the diagram of one Qwen3-8B layer, drawn from its configuration.

The table, row by row

- Architecture: encoder-decoder in 2017, decoder only today.
- Layers: 6 + 6 versus 36.
- Vector size: 512 versus 4,096.
- Attention: 8 classic heads versus 32 query heads for 8 key and value heads (GQA), with QK-Norm, a normalization of the queries and keys that stabilizes training.
- Feed-forward: ReLU and 2,048 hidden neurons versus SwiGLU and 12,288.
- Normalization: LayerNorm after each sub-block versus RMSNorm before each sub-block.
- Position: sinusoidal encoding added to the input versus RoPE in the attention, with a base of 10■.
- Vocabulary: about 37,000 tokens versus 151,669.
- Parameters: 65 million versus 8.2 billion, i.e. 125 times more.

The diagram, from bottom to top

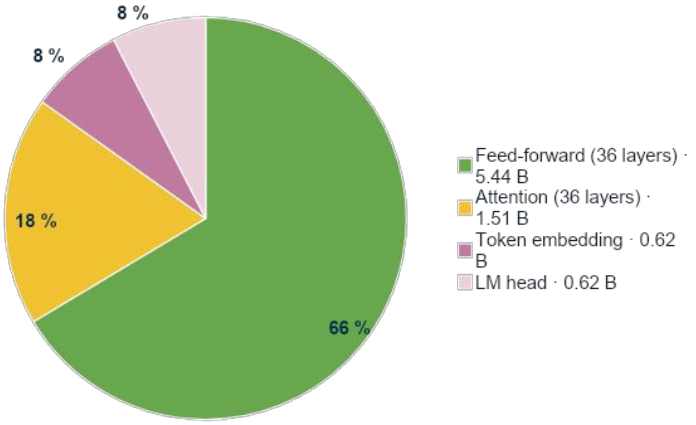
The embedding turns each token into a vector of 4,096. In each of the 36 layers: RMSNorm, then GQA attention (with RoPE and QK-Norm), then addition to the residual stream; RMSNorm, then SwiGLU feed-forward, then addition. The arrows on the left are the residual connections, which bypass each sub-block. After the last layer, a final RMSNorm, then the output head (LM head) gives a score for each of the 151,936 tokens, turned into next-token probabilities.

Message

The 2017 skeleton is intact. The changes are touch-ups, almost all motivated by training stability or efficiency. What has mostly changed is scale.

8.2 billion parameters ?!

Parameter count : computed from the configuration: d = 4,096, 36 layers, feed-forward 12,288, 32 + 8 heads of 128, vocabulary 151,936.



One layer	Shape	Parameters
q_proj	4,096 × 4,096	16.8 M
k_proj	4,096 × 1,024	4.2 M
v_proj	4,096 × 1,024	4.2 M
o_proj	4,096 × 4,096	16.8 M
gate_proj, up_proj	2 × 4,096 × 12,288	100.7 M
down_proj	12,288 × 4,096	50.3 M
Total		**193 M**

Two thirds of the parameters sit in the feed-forward layers.

What the slide shows

The breakdown of the 8.2 billion parameters of Qwen3-8B, computed from its configuration. On the left, a pie chart by main block; on the right, the detail of a single layer.

The pie chart

- Feed-forward, over the 36 layers (green): 5.44 billion, i.e. 66 %.
- Attention, over the 36 layers (yellow): 1.51 billion, i.e. 18 %.
- Input embedding (purple): 0.62 billion, i.e. 8 %.
- Output head, LM head (pale pink): 0.62 billion, i.e. 8 %. In Qwen3-8B, it is a matrix separate from the embedding.
- The normalizations weigh only about 0.3 million parameters, invisible at this scale.

The table of one layer

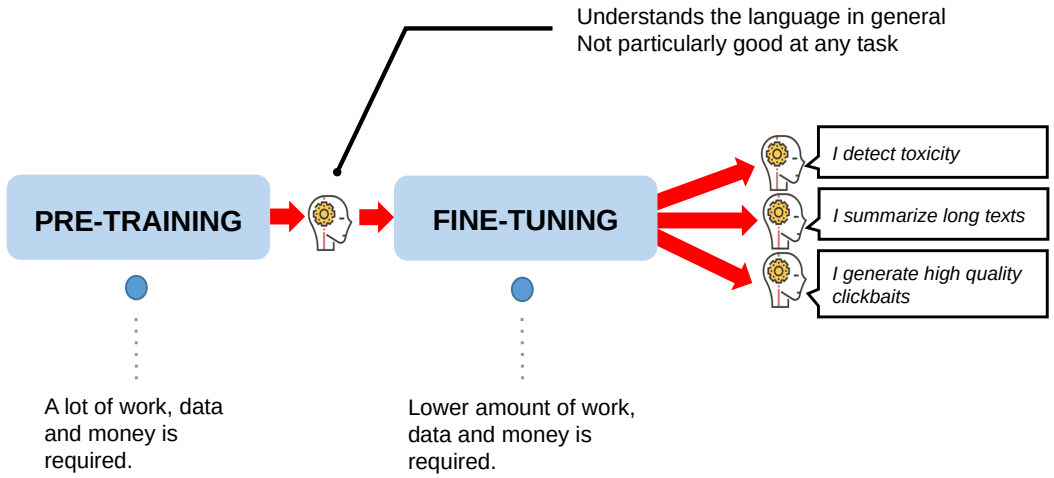
- q_proj: 4,096 × 4,096 = 16.8 M (32 heads of 128).
- k_proj and v_proj: 4,096 × 1,024 = 4.2 M each (8 heads of 128).
- o_proj, which mixes the heads after concatenation: 16.8 M.
- gate_proj and up_proj, the two SwiGLU projections to 12,288: 100.7 M together.
- down_proj, which brings it back to 4,096: 50.3 M.
- Total: about 193 M per layer.

What to notice

- Two thirds of the parameters are in the feed-forward, which explains why it is seen as the "memory" of the model, and why MoEs (next slides) choose to make this block sparse.
- k_proj and v_proj are 4 times smaller than q_proj: this is the direct trace of GQA, 8 key and value heads instead of 32.
- The names in the table are the actual names of the matrices in the Hugging Face code: we will meet them again in the LoRA part.

Pre-training, then post-training

Foundation model : pre-trained once on broad data , then adapted to many tasks .



What the slide shows

The complete two-stage training pipeline. On the left, pre-training: it requires a lot of work, data and money. It produces a model (the head in the middle) that "understands language in general, but is not particularly good at any task". On the right, fine-tuning, much cheaper, specializes this model for different tasks: detecting toxicity, summarizing long texts… or generating quality clickbait titles.

The definition

A foundation model is pre-trained once on very broad data, which is expensive, then adapted to many tasks, which costs much less.

Why two stages

The basic skill, understanding and producing language, is common to all tasks. It would be absurd to relearn it from scratch for each project. It is learned once, on trillions of tokens, then reused. This is the principle of transfer learning, already common in computer vision.

Orders of magnitude

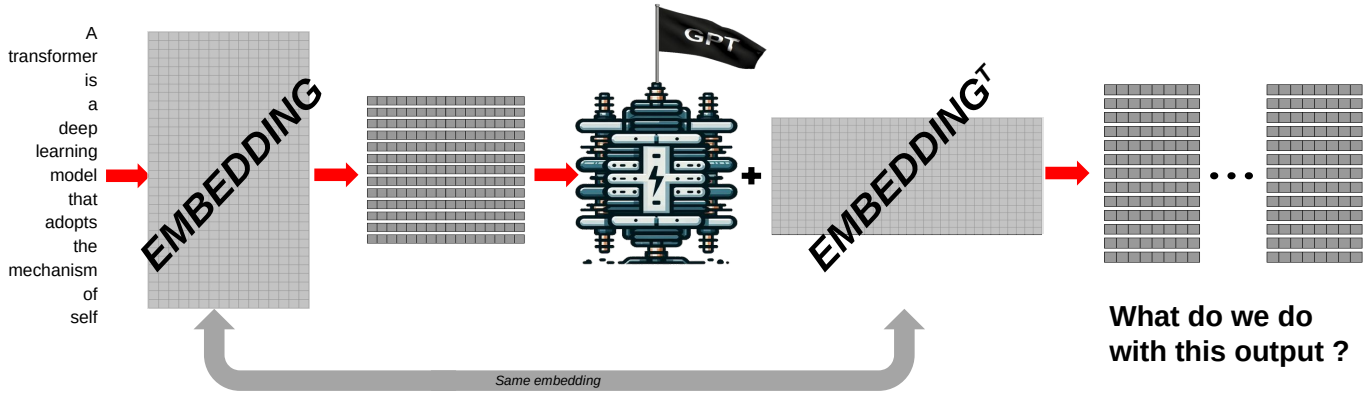
Pre-training a large model takes thousands of GPUs for weeks or months. A fine-tuning can be done on a single GPU in a few hours, with a few thousand examples.

Vocabulary

The title speaks of post-training: this is the term used today for everything that follows pre-training. It includes instruction fine-tuning, alignment with human preferences and reinforcement learning, presented in the second half of this part.

Pre-training a GPT-style model

Self-supervised learning the labels come from the text itself: the target of each position is the next token.



What the slide shows

The pre-training of a GPT-type model, from left to right. The text "A transformer is a deep learning model that adopts the mechanism of self" is split into tokens, goes through the embedding matrix (EMBEDDING), then through the Transformer (the GPT robot). At the output, each vector is multiplied by the transpose of the embedding matrix (EMBEDDING■), giving one vector of scores per position. The "Same embedding" arrow indicates that here, the same matrix is used at the input and the output. The question "What do we do with this output?" introduces the next slide.

The definition

Learning is self-supervised: the labels come from the text itself. The target of each position is simply the next token, which makes it possible to use any text without human annotation.

The output matrix

At the input, the matrix turns a token into a vector. At the output, its transpose compares the vector produced by the model with all the vocabulary tokens, by dot product, to get one score per token. Reusing the same matrix is called "weight tying": it saves a lot of parameters and is mostly done on small models.

In Qwen3-8B

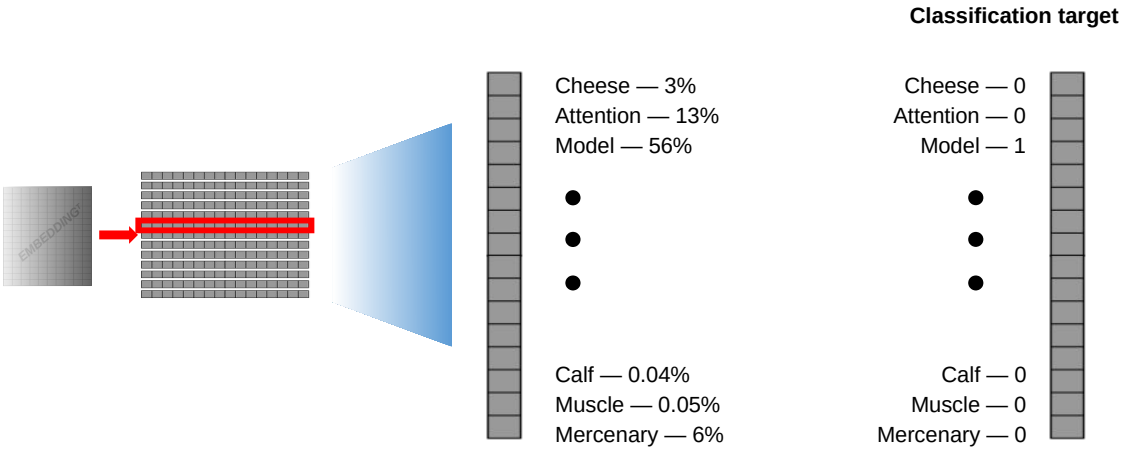
The two matrices are separate: an input embedding and an output head (LM head) of 151,936 × 4,096 each, i.e. about 622 million parameters each. The small Qwen3 models (0.6 B, 1.7 B and 4 B) tie their weights.

What is trained

Both ends are trained, like all the Transformer layers, by backpropagation of the prediction error.

From output vector to next-token probabilities

Cross-entropy loss : −log of the probability given to the correct token



What the slide shows

What is done with the output vector for a given position. The vector produced by the model (left) is multiplied by the output matrix, giving a score for each vocabulary token, then a softmax turns these scores into probabilities: Model 56 %, Attention 13 %, Mercenary 6 %, Cheese 3 %, Muscle 0.05 %, Calf 0.04 %… On the right, the classification target: a vector that is 1 for the correct token ("Model") and 0 everywhere else.

Key points

Next-token prediction is a simple classification problem, with as many classes as there are tokens in the vocabulary: 151,936 for Qwen3-8B.

The definition

The cross-entropy loss is –log of the probability given to the correct token. Here, –log(0.56) ≈ 0.58. If the model had given 99 % to "Model", the loss would be almost zero (0.01); if it had given 1 %, it would be about 4.6. The model is therefore penalized all the more when it was confident and wrong.

Training

This loss is computed at each position of each sentence in the batch, averaged, then backpropagation adjusts all the parameters to increase the probability of the correct tokens.

Vocabulary

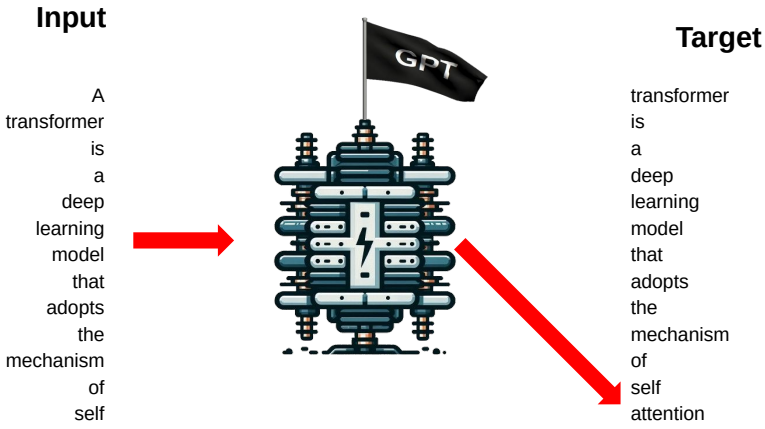
The scores before the softmax are called "logits".

At generation time

The same probabilities are used to choose the next token: either the most likely one, or a random draw weighted by these probabilities, which makes the answers more varied.

Next-token prediction

Next-token prediction the target is the input shifted by one, all positions are trained in parallel.



Next word prediction

What the slide shows

The pre-training task of a GPT-type model: next-token prediction. On the left, the input: "A transformer is a deep learning model that adopts the mechanism of self". On the right, the target: the same sentence shifted by one, "transformer is a deep learning model that adopts the mechanism of self attention".

How to read it

Each position must predict the next token: after "A", predict "transformer"; after "A transformer", predict "is"; and so on up to "attention" after "… of self".

The definition

The target is the input shifted by one position. Thanks to the causal mask, all positions are trained in parallel, in a single pass over the sentence, without any position seeing the answer.

What it brings

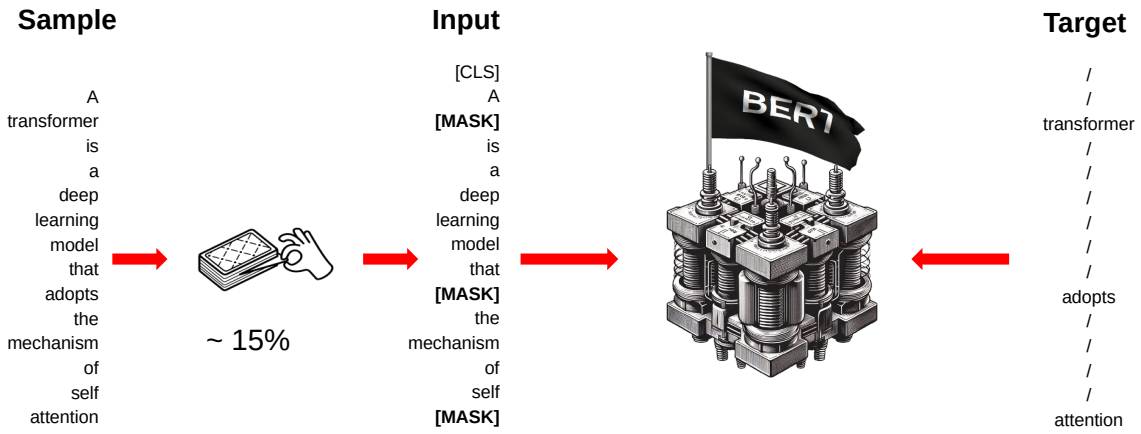
- No human annotation: any text provides both the inputs and the answers, which makes it possible to use trillions of tokens.
- A sentence of n tokens provides n – 1 training examples at once.
- The task is simple to state but very rich: to predict the next word well in every possible text, the model must learn grammar, factual knowledge, and part of reasoning.

At generation time

The model does exactly the same thing, but one token at a time: it predicts a token, appends it to the sequence, then predicts the next one.

Pre-training a BERT-style model

Masked language modeling : about 15 % of the tokens are hidden and the model must recover them. *(Devlin et al., 2019)*



Masked words prediction

What the slide shows

The pre-training of BERT, which is different because it is an encoder. On the left, the original sentence. In the center, about 15 % of the tokens are replaced by a special [MASK] token (here "transformer", "adopts" and "attention"), and a [CLS] token is added at the beginning. On the right, the target: the original words at the masked positions, and "/" everywhere else.

The definition

Masked language modeling consists of hiding about 15 % of the tokens and having the model recover the original words (Devlin et al., 2019).

Why not next-word prediction

An encoder sees the whole sentence in both directions: if it had to predict the next word, it would see it directly. Words are therefore hidden, and the model must guess them from the context on the left and on the right. It is a fill-in-the-blanks exercise, which takes full advantage of bidirectional attention.

Reading the target

The "/" marks the non-masked positions: no loss is computed on them. Only the masked positions serve as examples.

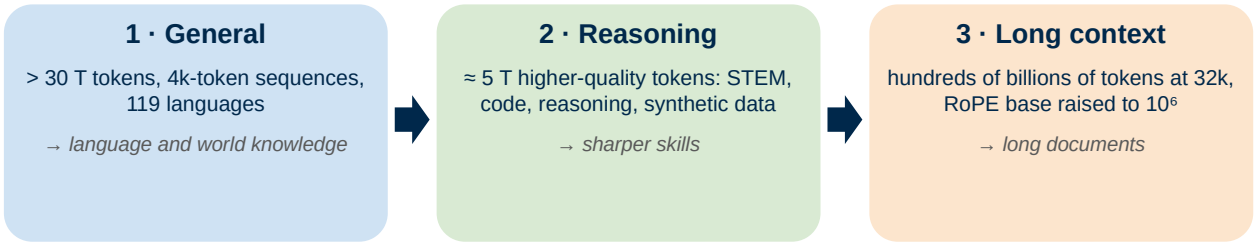
The [CLS] token

Added at the beginning of each sentence, its output vector is used to represent the whole sentence, for example for a classification task.

Drawback

Only 15 % of the positions serve as examples, versus almost all of them for GPT: each sentence brings less signal. In the paper, some of the chosen tokens are not replaced by [MASK] but by a random word or left unchanged, so that the model does not depend on the presence of the [MASK] token.

Pre-training Qwen3: data and phases



All Qwen3 models, from 0.6 B to 235 B parameters, see the same ≈ 36 T tokens. **Post-training comes next.**

Qwen3 Technical Report (arXiv:2505.09388), Section 3.

What the slide shows

The three phases of Qwen3 pre-training, as described in its technical report.

The three phases

- Phase 1, general: more than 30 trillion tokens, sequences of 4,096 tokens, 119 languages and dialects. The model learns language and general knowledge about the world.
- Phase 2, reasoning: about 5 trillion higher-quality tokens, with more science, code, reasoning and synthetic data produced by other models. The model refines its skills.
- Phase 3, long context: a few hundred billion tokens in sequences of 32,768 tokens, with the RoPE base raised from 10,000 to 1,000,000. The model learns to process long documents.

The definition

Pre-training data is trillions of tokens of filtered and deduplicated text. Its quality matters as much as its quantity.

What "filtered" covers

Quality classifiers, duplicate removal, removal of toxic content and personal data, balancing between web, code, mathematics and languages. Phases 2 and 3 are sometimes called "mid-training": the objective is still next-token prediction, but the data mix and the sequence length change.

What to notice

All Qwen3 models, from 0.6 to 235 billion parameters, see the same 36 trillion tokens. What then sets them apart is their size and their post-training.

Source

Qwen3 Technical Report (arXiv:2505.09388), section 3.



Post-training

Raw foundation models

Base model : the raw pre-trained model: very capable, but it only continues text.



Very powerful
Huge potential





Hard to control
Poor performance (on specific tasks)

Foundation model = Chaotic monster

What the slide shows

The raw foundation model, presented as a "chaotic monster" (a Cthulhu-inspired creature). On the left, its strengths: very powerful, huge potential. On the right, its weaknesses: hard to control, not very good at specific tasks.

The definition

The base model is the raw model that comes out of pre-training: very capable, but it only continues text.

Why it is powerful

It has "read" trillions of tokens. It knows the grammar of many languages, a large amount of facts, code, and forms of reasoning.

Why it is hard to control

It has learned only one thing: to continue a text in the most plausible way. It does not know that it is being asked a question, nor that a useful, polite or safe answer is expected. The result depends heavily on how the initial text is worded.

In Qwen3

The base models are released separately, with the suffix "Base" (for example Qwen3-8B-Base). The Qwen3-8B model used in this course is the post-trained version, which follows instructions. The base versions are mostly used as a starting point for further training.

What comes next

The next slide shows a concrete example of this behavior, then the post-training that turns this base model into an assistant.

Example of chaos

Instruction following — not learned by pre-training alone: the base model continues the review instead of rating it.



What the slide shows

An example of "chaos" with a base model. We write "My tailor is", and the model completes with "rich!": perfect, it is a famous sentence from English textbooks, very common in texts. Then we ask it: "Great, now, assign a number of stars to this film review: Star Wars The Last Jedi is really bad!". Instead of giving a rating, the model continues the review: "The story is awful. And so are characters. What a boring pile of garbage!".

The definition

Following instructions is not learned by pre-training alone: the base model continues the review instead of rating it.

The explanation

For a base model, the most likely continuation of a negative review is even more negative review. It does not distinguish between an instruction and a text to continue.

The solutions

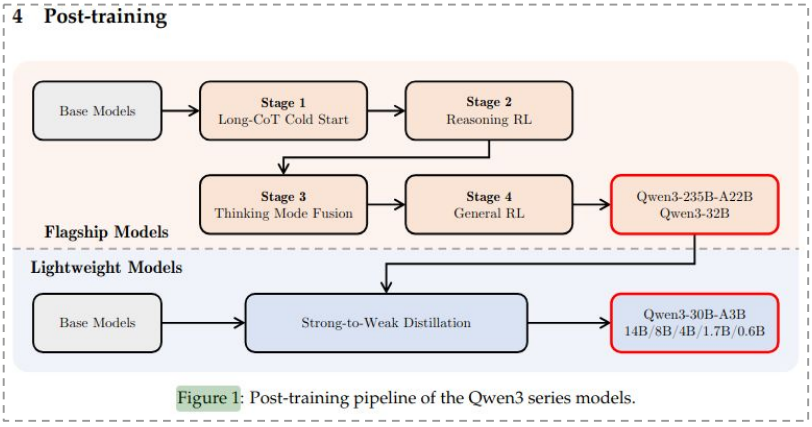
- Phrase the request in a form that the model will continue correctly, for example "Review: … Rating (1 to 5):". This is prompt engineering applied to a base model.
- Or, much more effective, post-train it so that it learns to recognize an instruction and respond to it. This is what distinguishes an "Instruct" or "chat" model from a base model.

Takeaway

The assistants we use every day are all base models that have been post-trained.

Post-training: from base model to assistant

Post-training : fine-tuning on instructions (SFT), then preference alignment and reinforcement learning.



Flagship models, 4 stages

- Long chain-of-thought SFT
- Reasoning RL (GRPO)
- Thinking-mode fusion
- General RL

Qwen3 Technical Report, Figure 1https://arxiv.org/abs/2505.09388

What the slide shows

The post-training of Qwen3. On the left, the placeholder for figure 1 of the technical report, the post-training pipeline of the Qwen3 series. On the right, the four stages followed by the flagship models, and the Qwen3-8B box.

The definition

Post-training turns the base model into an assistant: fine-tuning on instructions (SFT), then alignment with preferences and reinforcement learning.

The four stages of the flagship models (Qwen3-235B-A22B and Qwen3-32B)

- SFT on long chains of reasoning: the model learns to reason step by step on mathematics, code and logic problems.
- Reinforcement learning on reasoning, with the GRPO algorithm: the model generates several answers, and those verified as correct are reinforced.
- Mode fusion: the same model learns to answer with or without a thinking phase, which gives the "thinking" and "non-thinking" modes.
- General reinforcement learning, on more than 20 tasks (following instructions, formats, tool use), with rule-based or model-based rewards.

The figure

It shows these four stages starting from the base model, then the distillation to the smaller models.

The Qwen3-8B box

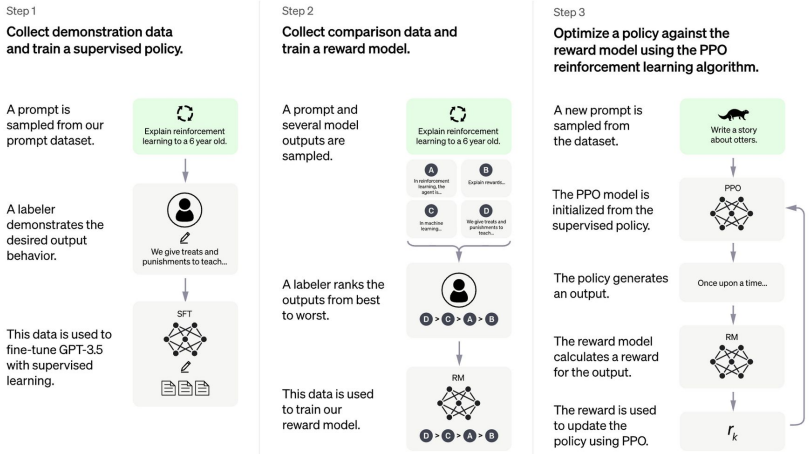
The small models, including Qwen3-8B, do not go through these four stages. They are obtained by "strong-to-weak" distillation: they are trained to reproduce the answers and then the output probabilities of the large models (Qwen3-32B and Qwen3-235B-A22B). This costs about ten times fewer GPU hours than the four full stages.

Source

Qwen3 Technical Report (arXiv:2505.09388), section 4.

Alignment: RLHF and DPO

Alignment : making the model's answers match human preferences: helpful, honest and harmless.



RLHF: the three steps on the left (Ouyang et al., 2022).

DPO (Rafailov et al., 2023): learns directly from pairs of preferred / rejected answers. No reward model, no RL.

RL with verifiable rewards: maths and code answers checked automatically. This is how reasoning models are trained (DeepSeek-R1, Qwen3).

Figure: OpenAI (2022), RLHF pipeline of InstructGPT / ChatGPT (Ouyang et al., arXiv:2203.02155), via github.com/opendilab/awesome-RLHF.

What the slide shows

On the left, the three-step RLHF diagram used for InstructGPT and ChatGPT. On the right, three alignment methods.

The definition

Alignment consists of making the model's answers match human preferences: helpful, honest and harmless.

RLHF, the three steps of the diagram (Ouyang et al., 2022)

- Step 1: annotators write the expected answer for prompts, and the model is fine-tuned on these examples (SFT).
- Step 2: the model produces several answers for the same prompt, annotators rank them from best to worst, and a reward model is trained to predict this ranking.
- Step 3: the model is optimized by reinforcement learning (PPO algorithm) to maximize the reward, while preventing it from drifting too far from the starting model.

DPO (Rafailov et al., 2023)

It starts from the same "preferred answer / rejected answer" pairs, but optimizes the model directly with a simple classification loss. There is no longer a separate reward model or reinforcement learning: it is much simpler and more stable, and widely used for open models.

Reinforcement learning with verifiable rewards

For mathematics and code, the correct answer can be checked automatically (a result, unit tests). The model generates several chains of reasoning, and those that reach the correct answer are reinforced. This is how reasoning models such as DeepSeek-R1 and Qwen3 are trained.

Source

Figure: OpenAI (2022), RLHF pipeline of InstructGPT / ChatGPT (Ouyang et al., arXiv:2203.02155), via github.com/opendilab/awesome-RLHF.